第九章

查 找

何谓查找表？

查找表是由**同一类型**的数据元素（或记录）构成的**集合**。

由于“集合”中的数据元素之间存在着松散的关系，因此查找表是一种应用灵便的结构。

对查找表经常进行的操作:

1）查询某个"特定的"数据元素是否在查找表中；

2）检索某个"特定的"数据元素的各种属性；

3）在查找表中插入一个数据元素；

4）从查找表中删去某个数据元素。

查找表可分为两类：

- **静态查找表**

  仅作查询和检索操作的查找表。

- **动态查找表**

  有时在查询之后，还需要将“查询”结果为“不在查找表中”的数据元素插入到查找表中；或者，从查找表中删除其“查询”结果为“在查找表中”的数据元素。

关键字

是数据元素（或记录）中某个数据项的值，用以标识（识别）一个数据元素（或记录）。

若此关键字可以识别唯一的一个记录，则称之谓“主关键字”。

若此关键字能识别若干记录，则称之“次关键字”

查找

根据给定的某个值，在查找表中确定一个其关键字等于给定值的数据元素或（记录）。

若查找表中存在这样一个记录，则称“查找成功”。查找结果给出整个记录的信息，或指示该记录在查找表中的位置；

否则称“查找不成功”。查找结果给出“空记录”或“空指针”。

如何进行查找？

查找的方法取决于查找表的结构。

由于查找表中的数据元素之间不存在明显的组织规律，因此不便于查找。

为了提高查找的效率，需要在查找表中的元素之间人为地附加某种确定的关系，换句话说，用另外一种结构来表示查找表。

9.1 静态查找表

9.2 动态查找树表

9.3 哈希表

9.1 静态查找表

抽象数据类型**静态查找表**的定义如下：

ADT StaticSearchTable {

数据对象D： D是具有相同特性的数据元素的集合。**每个数据元素含有类型相同的关键字**，可唯一标识数据元素。

数据关系R： 数据元素同属一个集合。

基本操作 **P**：

Create(**&**ST, n);

Destroy(**&**ST);

Search(ST, key);

Traverse(ST, Visit());

} ADT StaticSearchTable

Create(&ST, n);

操作结果：构造一个含n个数据元素的静态查找表ST。

Destroy(&ST);

初始条件：静态查找表ST存在；

操作结果：销毁表ST。

Search(ST, key);

初始条件：静态查找表ST存在，key 为和查找表中元素的关键字类型相同的给定值；

操作结果：若 ST 中存在其关键字等于 key 的数据元素，则函数值为该元素的值或在表中的位置，否则为“空”。

Traverse(ST, Visit());

初始条件：静态查找表ST存在，Visit是对元素操作的应用函数；

操作结果：按某种次序对ST的每个元素调用函数Visit()一次且仅一次，一旦Visit()失败，则操作失败。

假设静态查找表的顺序存储结构为

```
typedef  struct {
   ElemType *elem;
        // 数据元素存储空间基址，建表时
        // 按实际长度分配，0号单元留空
   int      length;   // 表的长度
} SSTable;
```

数据元素类型的定义为：

```
typedef struct {
    keyType key;      // 关键字域
    …… ……            // 其它属性域
} ElemType , TElemType ;
```

一、顺序查找表

二、有序查找表

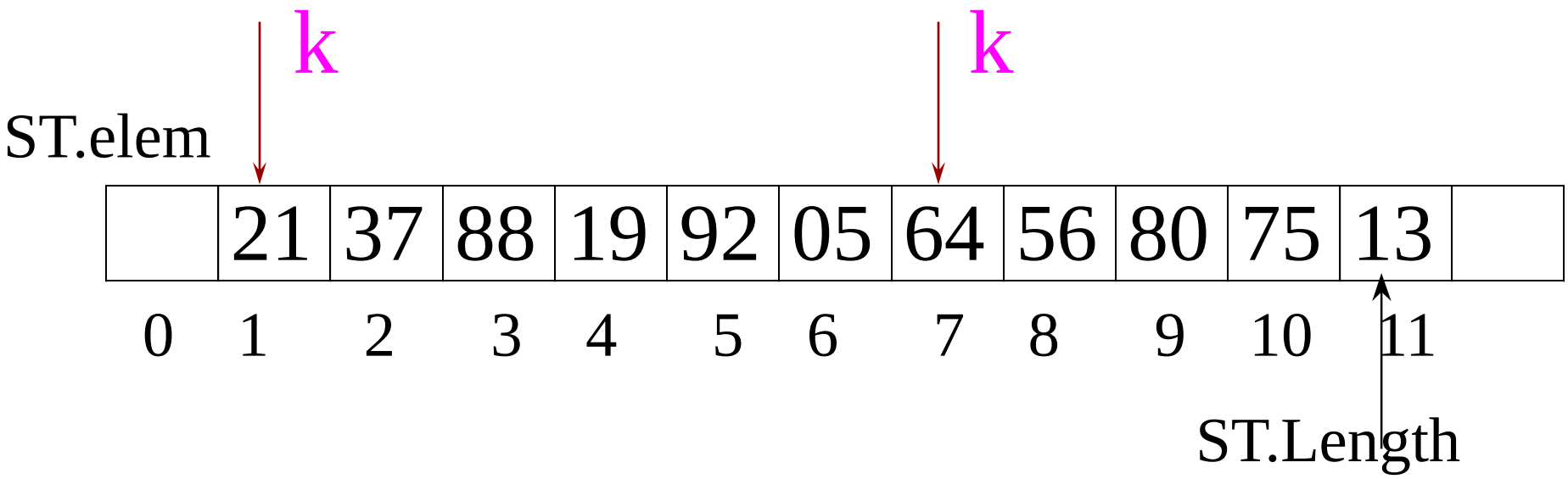
三、静态查找树表

四、索引顺序表

一、顺序查找表

以顺序表或线性链表表示静态查找表

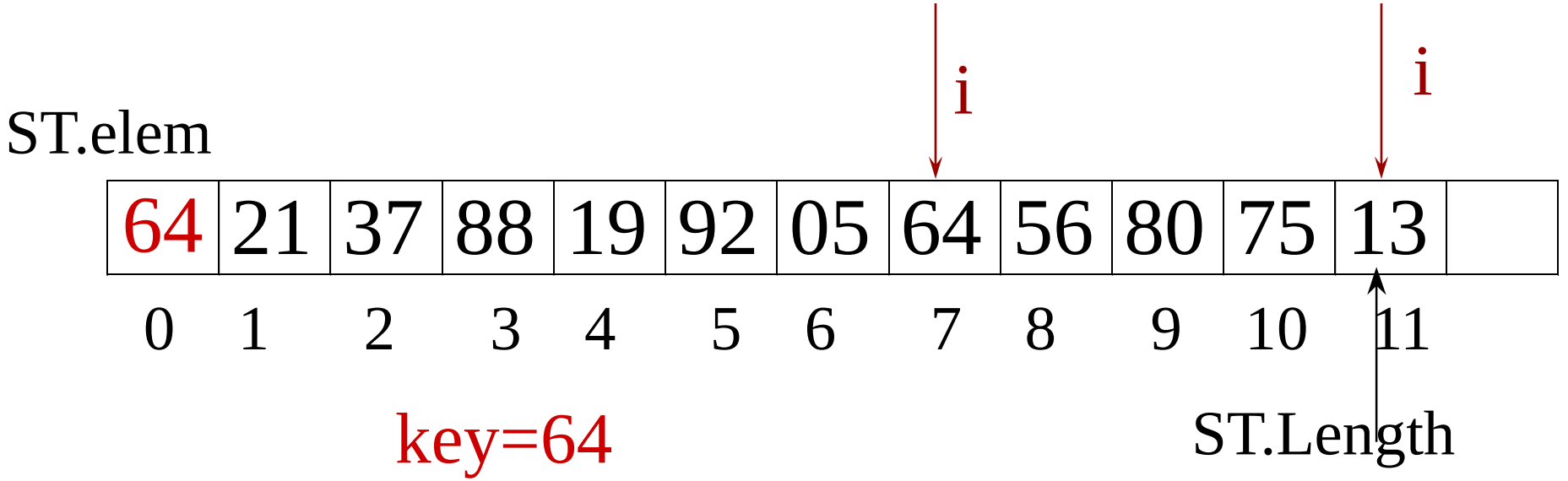
回顾顺序表的查找过程：

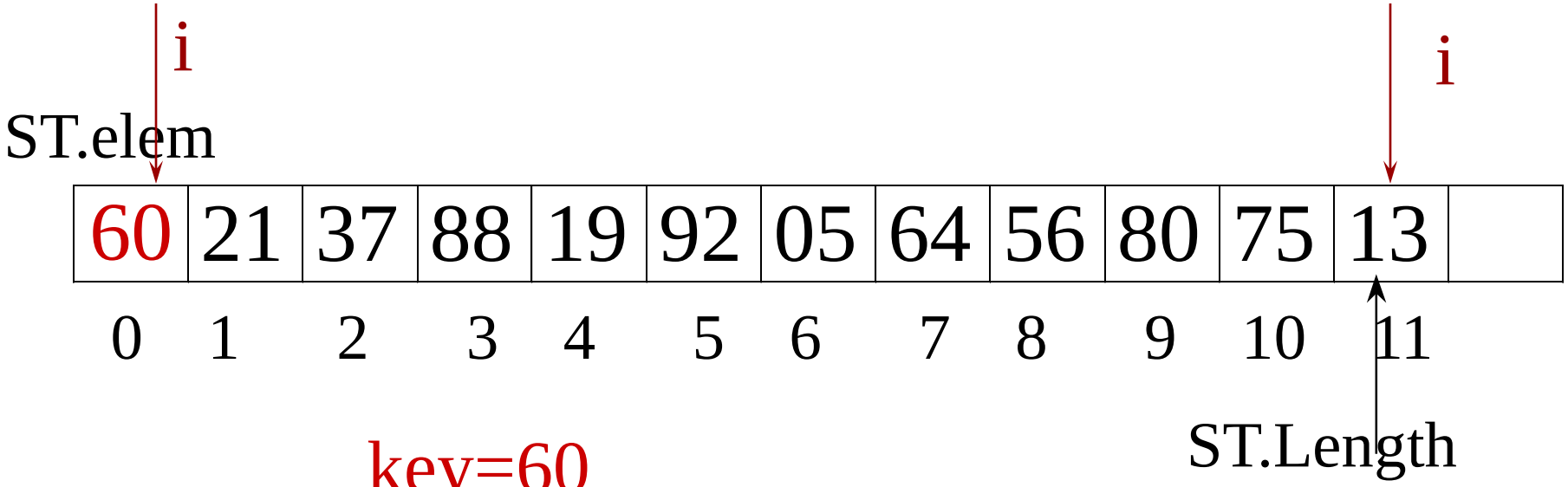


假设给定值 e=64,

要求 ST.elem[k] = e, 问: k = ?

```
int location( SqList L, ElemType& e,
        Status (*compare)(ElemType, ElemType)) {
    k = 1;
    p = L.elem;
    while ( k<=L.length &&
                !(*compare)(*p++,e)) k++;
    if ( k<= L.length) return k;
    else  return 0;
} //location
```


ST.elem
i
i
64 21 37 88 19 92 05 64 56 80 75 13
0 1 2 3 4 5 6 7 8 9 10 11
key=64
ST.Length


i
i
ST.elem
60 21 37 88 19 92 05 64 56 80 75 13
0 1 2 3 4 5 6 7 8 9 10 11
key=60
ST.Length

```
int Search_Seq(SSTable ST,
                                KeyType key) {
    // 在顺序表ST中顺序查找其关键字等于
    // key的数据元素。若找到，则函数值为
    // 该元素在表中的位置，否则为0。
  ST.elem[0].key = key;        // “哨兵”
  for (i=ST.length; ST.elem[i].key!=key; --i );
                               // 从后往前找
  return i;                    // 找不到时，i为0
} // Search_Seq
```

分析顺序查找的时间性能

定义： 查找算法的**平均查找长度**

(**A**verage **S**earch **L**ength)

为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值

$$ASL = \sum_{i=1}^{n} P_i C_i$$

其中：n 为表长，P_i 为查找表中第i个记录的概率，且 $\sum_{i=1}^{n} P_i = 1$，C_i 为找到该记录时，曾和给定值比较过的关键字的个数。

对顺序表而言，$C_i = n-i+1$

$$ASL = nP_1 + (n-1)P_2 + \ldots + 2P_{n-1} + P_n$$

在等概率查找的情况下，$P_i = \frac{1}{n}$

顺序表查找的平均查找长度为：

$$ASL_{ss} = \frac{1}{n}\sum_{i=1}^{n}(n-i+1) = \frac{n+1}{2}$$

在不等概率查找的情况下，ASL_{ss} 在

$$P_n \geqslant P_{n-1} \geqslant \cdots \geqslant P_2 \geqslant P_1$$

时取极小值

若查找概率无法事先测定，则查找过程采取的改进办法是，在每次查找之后，将刚刚查找到的记录直接移至表尾的位置上。

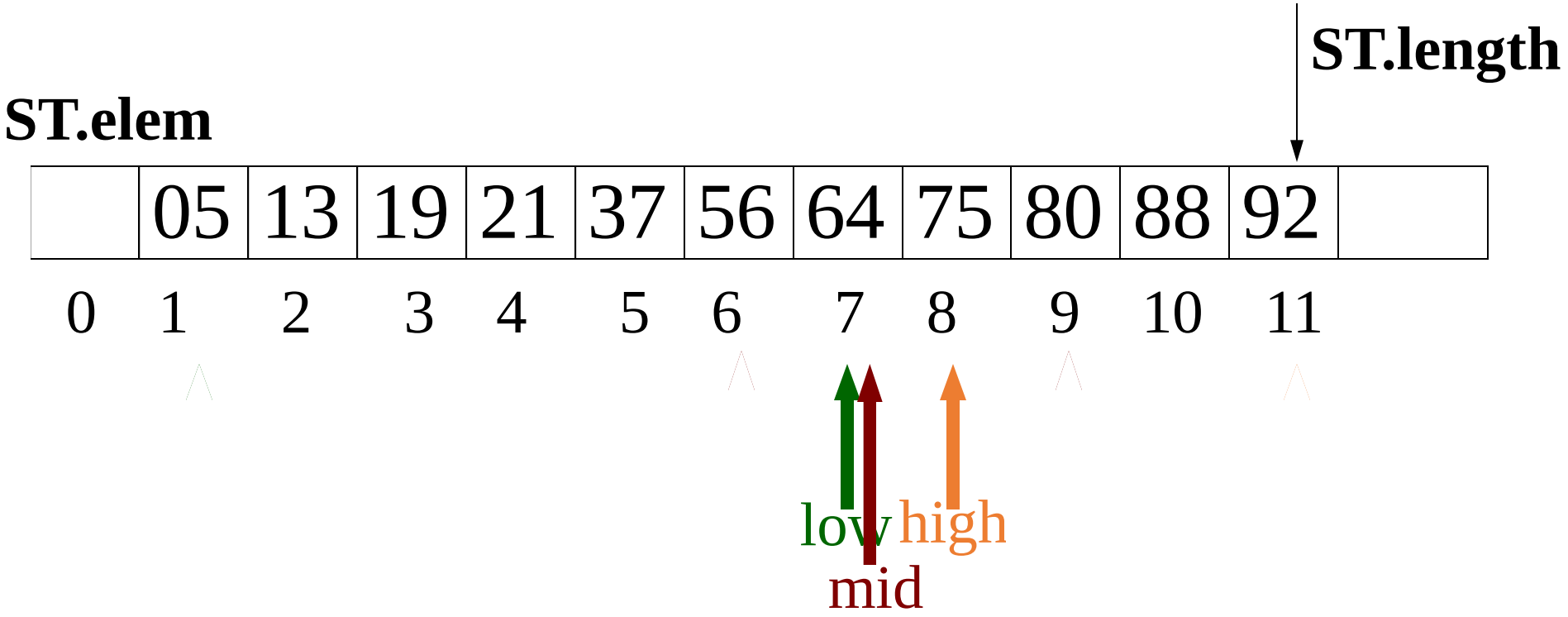
二、有序查找表

上述顺序查找表的查找算法简单，但平均查找长度较大，特别不适用于表长较大的查找表。

若以**有序表**表示静态查找表，则查找过程可以基于“**折半**”进行。

- 折半查找又称为二分查找
- 基本思想:如果查找表中的数据元素按关键字有序(假设递增有序),则在查找时不必逐个顺序比较,而采用跳跃的方式,
  - 先与“中间位置”的记录关键字值比较,若相等则查找成功;
  - 若给定值大于“中间位置”的关键字值,则在后半部继续进行折半查找;
  - 否则在前半部进行折半查找.
- 折半查找仅适用于以顺序存贮结构组织的查找表的查找。

例如: key=64 的查找过程如下：



low 指示查找区间的下界

high 指示查找区间的上界

mid = (low+high)/2

折半查找算法

设待查元素所在区域的下界为low,上界为hig,则中间位置mid=(low + hig)/2

- 若此元素关键字值等于给定值,则查找成功;
- 若此元素关键字值大于给定值,则在区域[mid+1, hig]进行折半查找
- 若此关键字值小于给定值,则在区域[low, mid-1]内进行折半查找

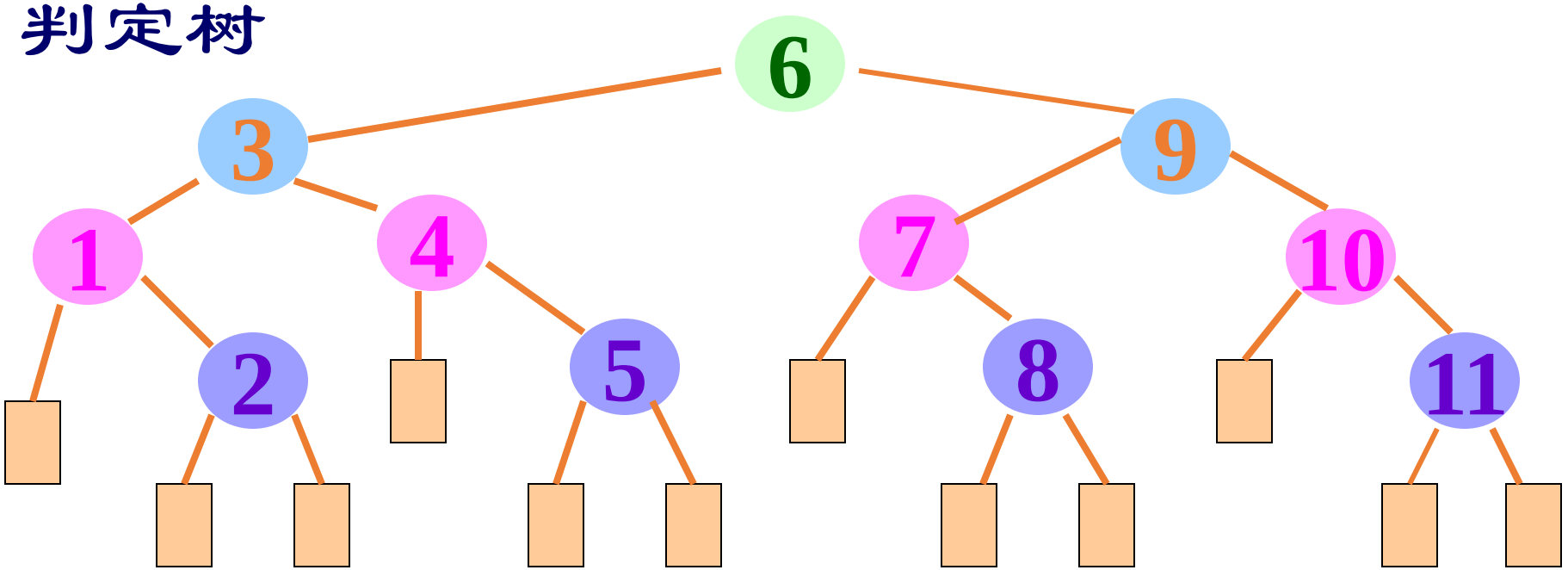
```
int Search_Bin ( SSTable ST, KeyType key ) {
   low = 1;  high = ST.length;     // 置区间初值
   while (low <= high) {
      mid = (low + high) / 2;
      if （EQ (key , ST.elem[mid].key) )
          return  mid;          // 找到待查元素
      else  if ( LT (key , ST.elem[mid].key) )
          high = mid - 1;       // 继续在前半区间进行查找
      else  low = mid + 1;   // 继续在后半区间进行查找
   }
   return 0;                  // 顺序表中不存在待查元素
} // Search_Bin
```

分析折半查找的平均查找长度

先看一个具体的情况，假设：n=11

i	1	2	3	4	5	6	7	8	9	10	11
Ci	3	4	2	3	4	1	3	4	2	3	4

判定树



一般情况下，表长为 n 的折半查找的判定树的深度和含有 n 个结点的完全二叉树的深度相同。

假设 $n=2^h-1$ 并且查找概率相等

则 $$ASL_{bs}=\frac{1}{n}\sum_{i=1}^{n}C_i=\frac{1}{n}\left[\sum_{j=1}^{h}j\times 2^{j-1}\right]=\frac{n+1}{n}\log_2(n+1)-1$$

在n>50时，可得近似结果

$$ASL_{bs}\approx\log_2(n+1)-1$$

三、静态查找树表

当有序表中各记录的查找概率相等时，折半查找其性能最优。

在不等概率查找的情况下，折半查找情况如何？

例：假设有序表中含5个记录，并且已知各记录的查找概率不等，

关键字:	A	B	C	D	E
Pi:	0.2	0.3	0.05	0.3	0.15
Ci:	2	3	1	2	3

此时ASL=2×0. 2+3×0. 3+1×0. 05+2×0. 3+3×0. 15=2. 4

若改变Ci的值：　2　1　3　2　3

则ASL=2×0. 2+1×0. 3+3×0. 05+2×0. 3+3×0. 15=1. 9

这就说明，当有序表中各记录的查找概率不等时，折半查找性能未必是最优的。

那么，描述此类查找过程的判定树为何类二叉树时，其查找性能最佳？

如果只考虑查找成功的情况，则使查找性能达到**最佳的判定树**是其带权内路径长度之和PH值

$$PH = \sum_{i=1}^{n} w_i h_i$$

取最小值的二叉树。

其中：n为二叉树上结点的个数；h_i为第i个结点在二叉树上的层次数；w_i为结点的权，并且$w_i=cp_i$ $(i=1, 2, \cdots, n)$，p_i为结点的查找概率，c为某个常量。

称PH值取最小的二叉树为**静态最优查找树**。

构造静态最优查找树花费的时间代价较高，在此不做讨论，

仅介绍构造近似最优查找树——次优查找树的有效算法。

次优查找树

已知一个按关键字有序的记录序列

$$(r_l, r_{l+1}, \cdots, r_h)$$

其中$r_l.key < r_{i+1}.key < \cdots < r_h.key$ 与每个记录相应的权值为w_l，w_{i+1}，$\cdots$，w_h

现构造一棵二叉树，使这棵二叉树的带权路径长度PH值在所有具有同样权值的二叉树中近似为最小，称这类二叉树为**次优查找树**。

次优查获树的构造方法：

首先取第i（$\ell \leqslant i \leqslant h$）个记录构造根结点$r_i$

使 $\Delta P_i = \left| \sum_{j=i+1}^{h} w_j - \sum_{j=1}^{i-1} w_j \right|$ 达最小

然后分别对子序列$(r_l, r_{l+1}, \ldots, r_{i-1})$和$(r_{i+1}, r_{i+2}, \ldots, r_h)$构造两棵次优查找树，并分别设为根结点$r_i$的左子树和右子树。

次优查获树的构造方法：

首先取第i（$\ell \leqslant i \leqslant h$）个记录构造根结点$r_i$

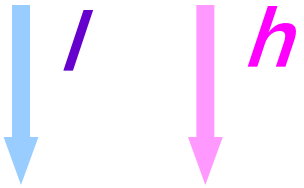
使 $\Delta P_i = \left| \sum_{j=i+1}^{h} w_j - \sum_{j=1}^{i-1} w_j \right|$ 达最小

为便于计算，引入累计权值和 $sw_i = \sum_{j=1}^{i} w_j$

并设 $w_{l-1} = 0$ 和 $sw_{l-1} = 0$，

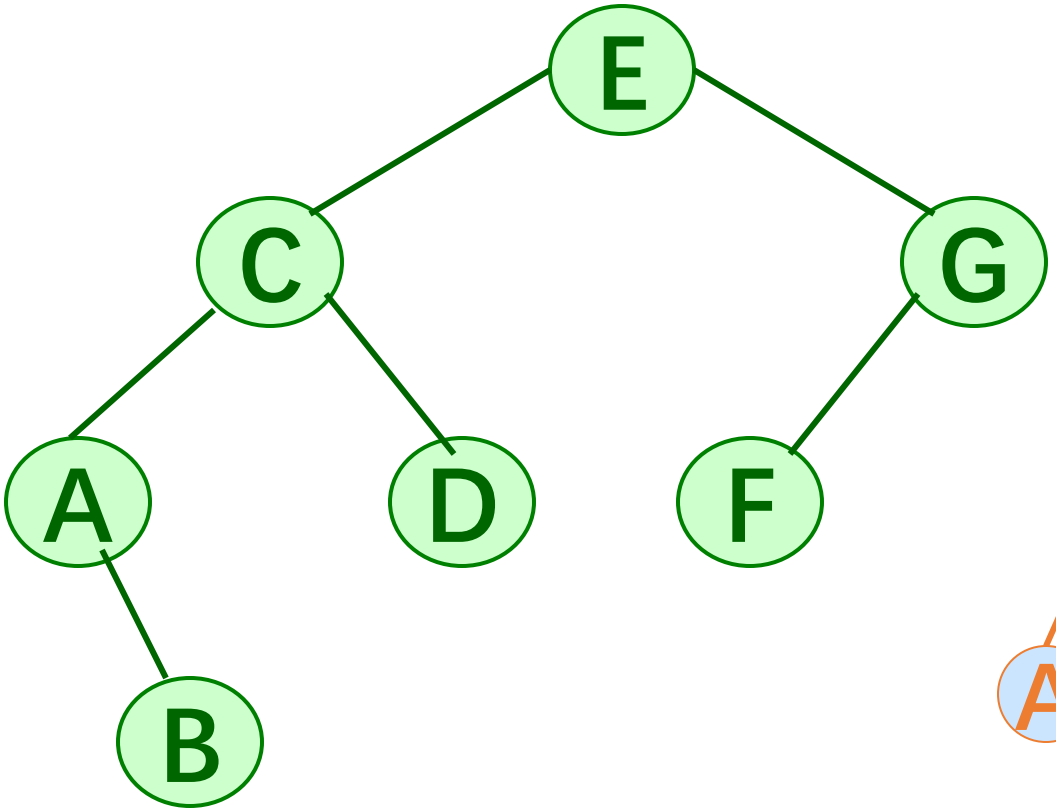
则推导可得 $\Delta P_i = \left| (sw_h + sw_{l-1}) - sw_i - sw_{i-1} \right|$

例如:

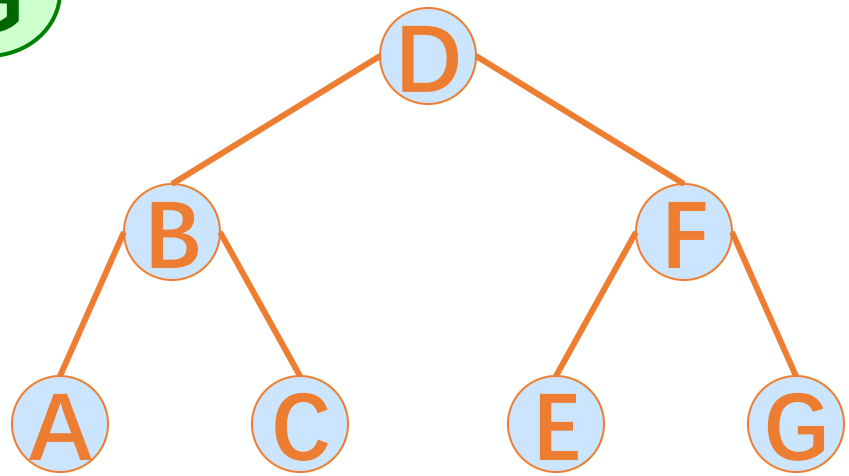


j	0	1	2	3	4	5	6	7
w_j	0	2	1	5	3	4	3	5
sw_j	0	2	3	8	11	15	18	23
Δp_j		21	18	12	4	3	10	18
		9	6	0	8		5	3
		1	2					
key		A	B	C	D	E	F	G

所得次优查找树如下所示:



和折半查找相比较



查找比较“总次数”
= 3×2+4×1+2×5+3×3
+1×4+3×3+2×5 = 52

查找比较“总次数”
= 3×2+2×1+3×5+1×3
+3×4+2×3+3×5 = 59

构造次优查找树的算法

```
Status SecondOptimal(BiTree &T, ElemType R[],
                     float sw[], int low, int high) {
  // 由有序表R[low..high]及其累计权值表sw
  // 递归构造次优查找树T。
  选择最小的ΔP_i值
  if (!(T = (BiTree)malloc(sizeof(BiTNode))))
    return ERROR;
  T->data = R[i];                    // 生成结点
```

```
if (i==low)  T->lchild = NULL;  // 左子树空
else  SecondOptimal(T->lchild, R, sw, low, i-1);
                                  // 构造左子树
 if (i==high)  T->rchild = NULL;  // 右子树空
 else  SecondOptimal(T->rchild, R, sw, i+1, high);
                                  // 构造右子树
 return OK;
} // SecondOptimal
```

次优查找树采用二叉链表的存储结构

```
Status CreateSOSTre(SOSTree &T, SSTable ST) {
  // 由有序表 ST 构造一棵次优查找树 T
  // ST的数据元素含有权域 weight
  if (ST.length = 0)  T = NULL;
  else {
     FindSW(sw, ST);
              // 按照有序表 ST 中各数据元素
              // 的 weight 值求累计权值表
     SecondOpiamal(T, ST.elem, sw, 1, ST.length);
  }
  return OK;
} // CreatSOSTree
```

四、索引顺序表

- 索引顺序查找又称分块查找
- 数据元素的特点：
  - ✓ 若把所有n个数据元素分成m块：
    - ① 第一块中任一元素的关键字都小于第二块中任一元素的关键字，
    - ② 第二块中任一元素的关键字都小于第三块中的任一元素的关键字...，
    - ③ 第m-1块中任一元素的关键字都小于第m块中的任一元素的关键字.
  - ✓ 每一块中元素的关键字不一定是有序的。

索引顺序查找算法

基本思想：

①先抽出各块中的最大关键字构成一个索引表；

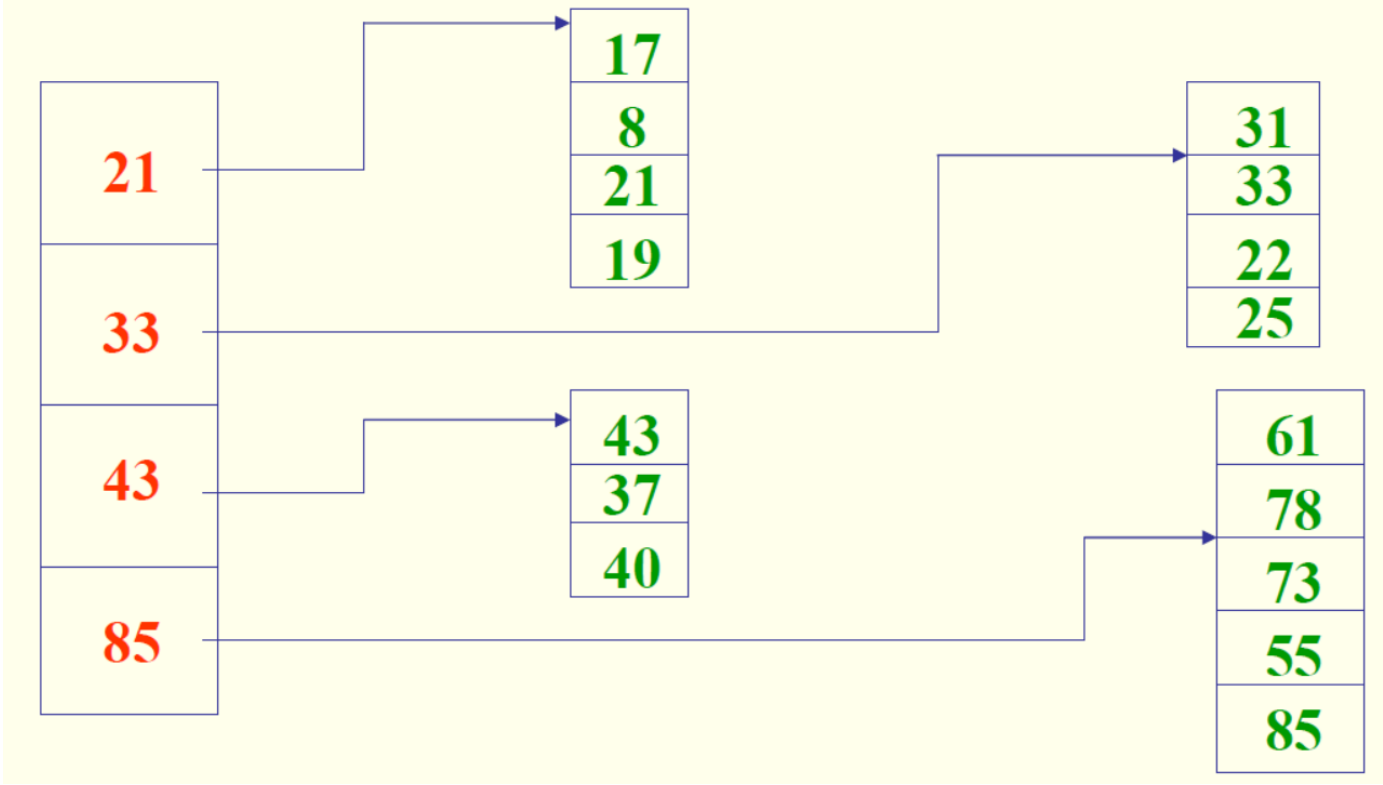
②查找分两部进行：

- 先对索引表进行折半查找或顺序查找，确定待查记录在哪一块。

- 在已确定的那一块中进行顺序查找。

索引顺序查找示例

将17, 8, 21, 19, 31, 33, 22, 25, 43, 37, 40, 61, 78, 73, 55, 85分为4块：[17, 8, 21, 19]， [31, 33, 22, 25]， [43, 37, 40]，[61, 78, 73, 55, 85]以每块中最大关键字作为该块所有元素的索引



索引顺序表的查找过程：

1）由索引确定记录所在区间；

2）在顺序表的某个区间内进行查找。

可见，**索引顺序查找**的过程也是一个

“缩小区间”的查找过程。

索引顺序查找的平均查找长度 =

查找“索引”的平均查找长度

+ 查找“顺序表”的平均查找长度

四种查找方法比较

	顺序查找	折半查找	静态树表查找	分块查找
平均查找长度	最大	等概率查找最小	不等概率查找最小	次小
表的结构	有序表、无序表均可	仅适用于有序表	仅适用于有序表	表中元素逐段有序
表的存储结构	顺序、链式结构均可	仅适用于顺序存储	仅适用于顺序存储	顺序、链式均可，索引表顺序存储

9.2 动态查找树表

抽象数据类型动态查找表的定义如下：

ADT DynamicSearchTable {

数据对象D： D是具有相同特性的数据元素的集合。每个数据元素含有类型相同的关键字，可唯一标识数据元素。

数据关系R： 数据元素同属一个集合。

基本操作P:

InitDSTable(&DT)

DestroyDSTable(&DT)

SearchDSTable(DT, key);

InsertDSTable(&DT, e);

DeleteDSTable(&T, key);

TraverseDSTable(DT, Visit());

}ADT DynamicSearchTable

InitDSTable(&DT);

操作结果：

构造一个空的动态查找表DT。

DestroyDSTable(&DT);

初始条件：动态查找表DT存在；

操作结果：销毁动态查找表DT。

SearchDSTable(DT, key);

初始条件：动态查找表DT存在，**key**为和关键字类型相同的给定值；

操作结果：若DT中存在其关键字等于**key**的数据元素，则函数值为该元素的值或在表中的位置，否则为“空”。

InsertDSTable(&DT, e);

初始条件：动态查找表DT存在，

e 为待插入的数据元素；

操作结果：若DT中不存在其关键字

等于 e.key 的 数据元素，

则插入 e 到DT。

DeleteDSTable(&DT, key);

初始条件：动态查找表DT存在，key为和关键字类型相同的给定值；

操作结果：若DT中存在其关键字等于key的数据元素，则删除之。

TraverseDSTable(DT, Visit());

初始条件：动态查找表DT存在，**Visit**是对结点操作的应用函数；

操作结果：按某种次序对DT的每个结点调用函数 **Visit()** 一次且至多一次。一旦 **Visit()** 失败，则操作失败。

综合上一节讨论的几种查找表的特性：

	查找	插入	删除
无序顺序表	O(n)	O(n)	O(n)
无序线性链表	O(n)	**O(1)**	**O(1)**
有序顺序表	**O(logn)**	O(n)	O(n)
有序线性链表	O(n)	**O(1)**	**O(1)**
静态查找树表	**O(logn)**	O(nlogn)	O(nlogn)

可得如下结论：

1）从查找性能看，最好情况能达O(logn)，此时要求表有序；

2）从插入和删除的性能看，最好情况能达O(1)，此时要求存储结构是链表。

一、二叉排序树（二叉查找树）

二、二叉平衡树

三、B - 树

四、B^+树

五、键　树



一、二叉排序树

(二叉查找树)

1. 定义

2. 查找算法

3. 插入算法

4. 删除算法

5. 查找性能的分析



1. 定义：

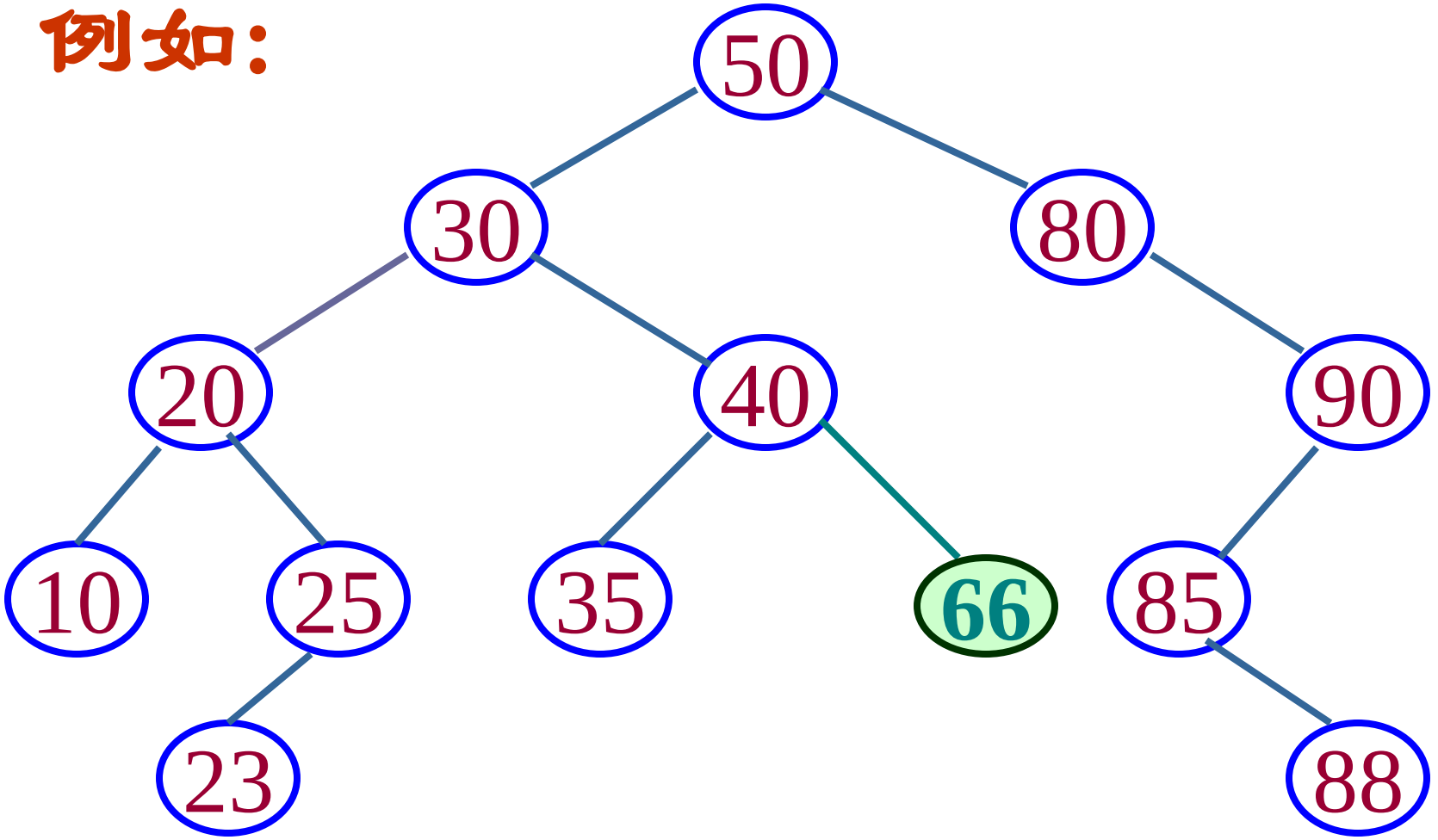
二叉排序树或者是一棵空树；或者是具有如下特性的二叉树：

（1）若它的左子树不空，则左子树上所有结点的值均小于根结点的值；

（2）若它的右子树不空，则右子树上所有结点的值均大于根结点的值；

（3）它的左、右子树也都分别是二叉排序树。

例如：



不是二叉排序树。

通常，取二叉链表作为二叉排序树的存储结构

```
typedef struct BiTNode { // 结点结构
    TElemType      data;
    struct BiTNode  *lchild, *rchild;
                                  // 左右孩子指针
} BiTNode, *BiTree;
```

2．二叉排序树的查找算法：

若二叉排序树为空，则查找不成功；

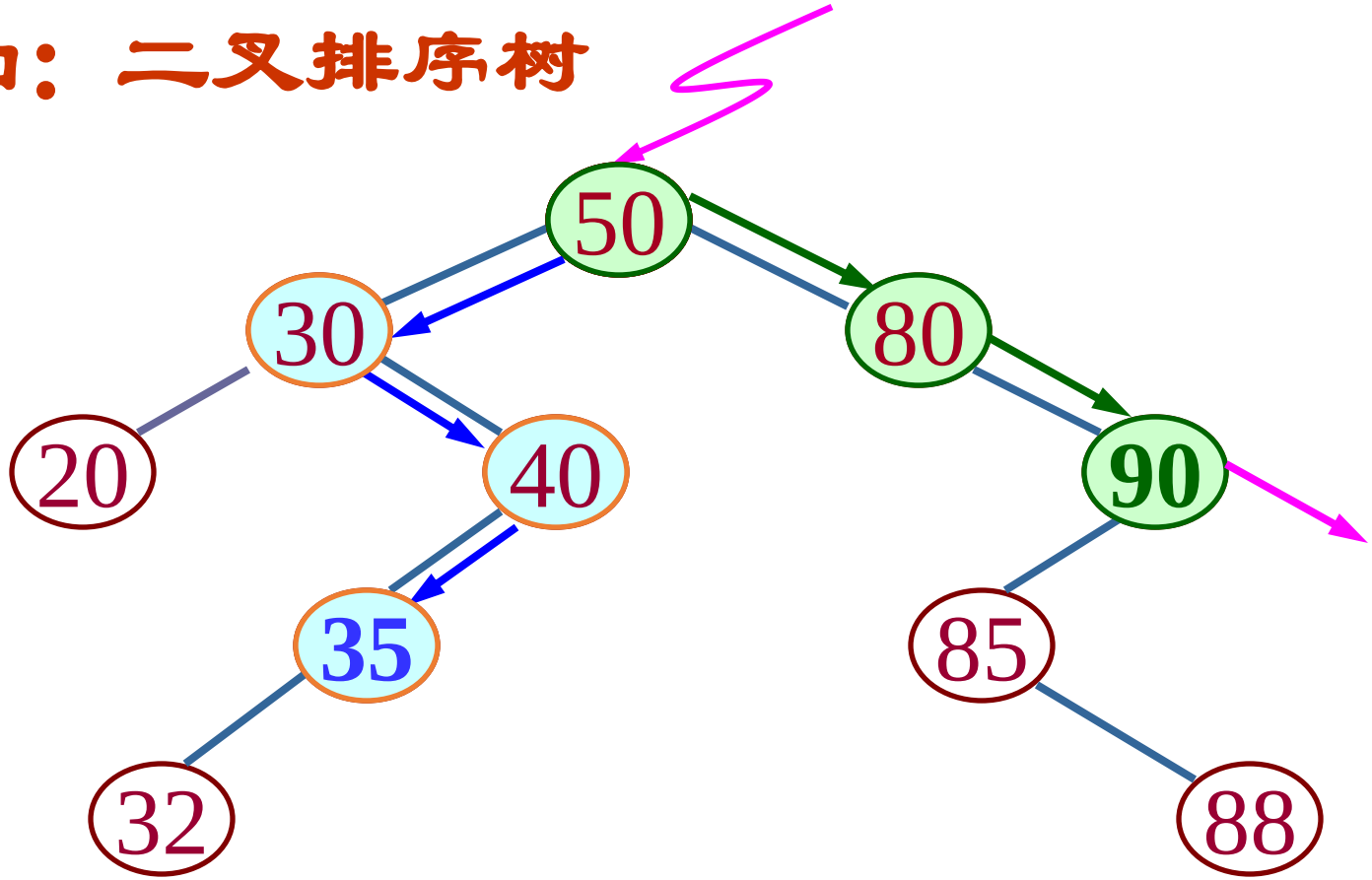
否则，

1）若给定值等于根结点的关键字，则查找成功；

2）若给定值小于根结点的关键字，则继续在左子树上进行查找；

3）若给定值大于根结点的关键字，则继续在右子树上进行查找。

例如：二叉排序树



查找关键字

== 50 , 35 , 90 , 95

从上述查找过程可见，

在查找过程中，生成了一条查找路径：

从根结点出发，沿着左分支或右分支逐层向下直至关键字等于给定值的结点；

——查找成功

或者

从根结点出发，沿着左分支或右分支逐层向下直至指针指向空树为止。

——查找不成功

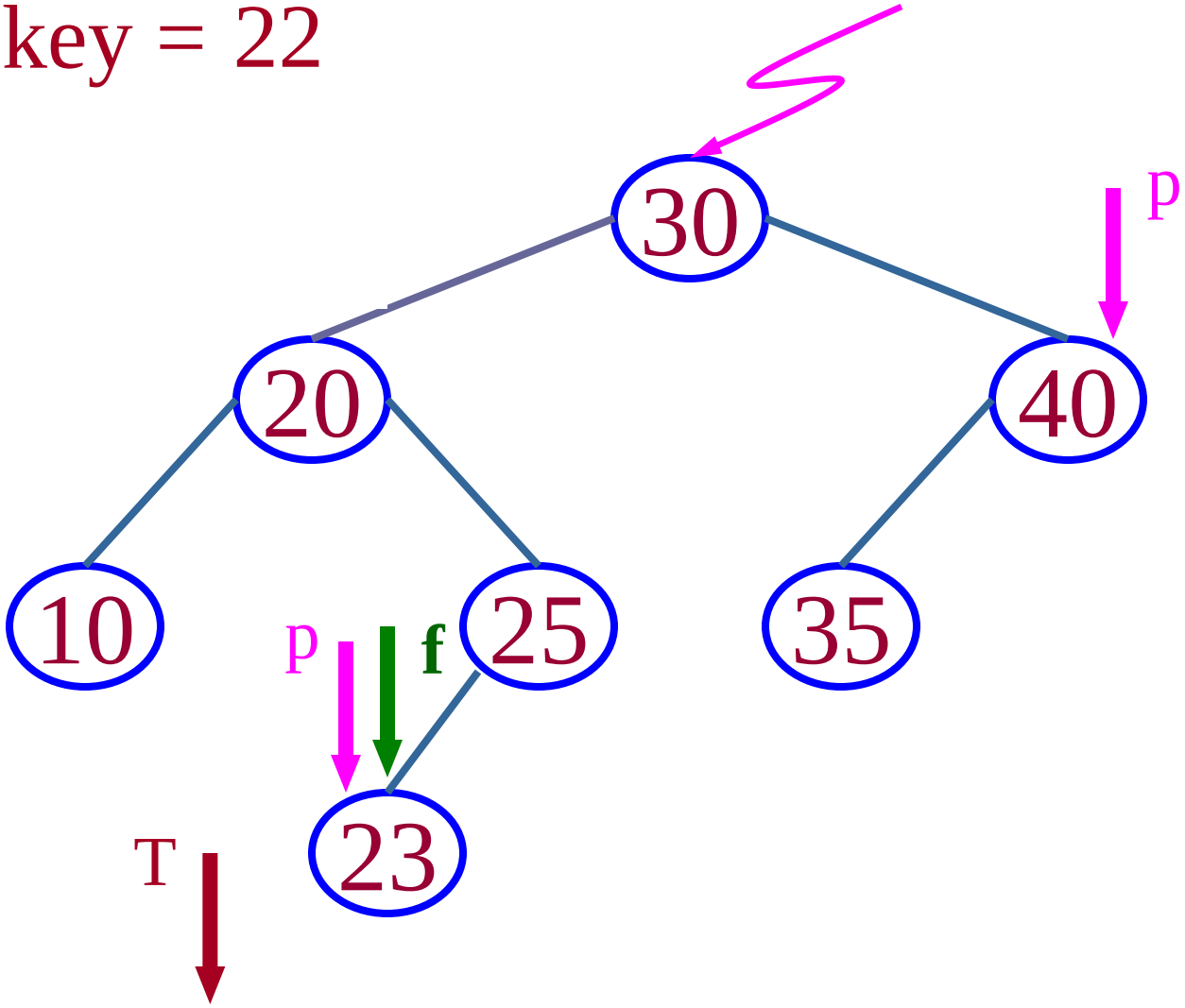
算法描述如下：

```
Status SearchBST (BiTree T, KeyType key,
                  BiTree f, BiTree &p ) {
  // 在根指针 T 所指二叉排序树中递归地查找其
  // 关键字等于 key 的数据元素，若查找成功，
  // 则返回指针 p 指向该数据元素的结点，并返回
  // 函数值为 TRUE;   否则查找不成功，返回
  // 指针 p 指向查找路径上访问的最后一个结点，
  // 并返回函数值为FALSE, 指针 f 指向当前访问
  // 的结点的双亲，其初始调用值为NULL
      ... ... ... ...

} // SearchBST
```

```
if (!T)
    { p = f;  return FALSE; }  // 查找不成功
else  if ( EQ(key, T->data.key) )
    { p = T;  return TRUE; }  // 查找成功
else  if ( LT(key, T->data.key) )
     SearchBST (T->lchild, key, T, p );
                          // 在左子树中继续查找
else  SearchBST (T->rchild, key, T, p );
                          // 在右子树中继续查找
```

设 key = 22



3. 二叉排序树的插入算法

根据动态查找表的定义，“插入”操作在查找不成功时才进行；

若二叉排序树为空树，则新插入的结点为新的根结点；否则，新插入的结点必为一个新的叶子结点，其插入位置由查找过程得到。

```
Status Insert BST(BiTree &T, ElemType e )
{
 // 当二叉排序树中不存在关键字等于 e.key 的
 // 数据元素时，插入元素值为 e 的结点，并返
 // 回 TRUE; 否则，不进行插入并返回FALSE
  if (!SearchBST ( T, e.key, NULL, p ))
    {      …    …        }
  else return FALSE;
} // Insert BST
```

```
s = (BiTree) malloc (sizeof (BiTNode));
                        // 为新结点分配空间
s->data = e;
s->lchild = s->rchild = NULL;

if  ( !p )  T = s;      // 插入 s 为新的根结点

else   if ( LT(e.key, p->data.key) )
       p->lchild = s;          // 插入 *s 为 *p 的左孩子
else  p->rchild = s;   // 插入 *s 为 *p 的右孩子

return TRUE;      // 插入成功
```

结论：

(1) 一个无序序列可以通过构造一棵二叉排序树而变成一个有序序列，构造树的过程即为对无序序列进行排序的过程；

(2) 每次插入的新结点都是二叉排序树的叶子结点，在进行插入操作时，不必移动其它结点，仅需修改某个结点的指针由空变为非空即可。这就相当于在一个有序序列上插入一个元素而没有移动其它元素。这个特性告诉我们，对于需要经常插入和删除记录的有序表采用二叉排序树结构更为合适。

第十一次作业：

设查找的关键字序列为：

{36, 20, 59, 32, 67, 18, 90, 48, 12, 75, 39, 15, 42}

画出通过逐个节点的插入操作所生成的二叉排序树

4. 二叉排序树的删除算法

和插入相反，删除在**查找成功**之后进行，并且要求在删除二叉排序树上某个结点之后，**仍然保持二叉排序树的特性**。

可分三种情况讨论：

（1）被删除的结点是叶子；

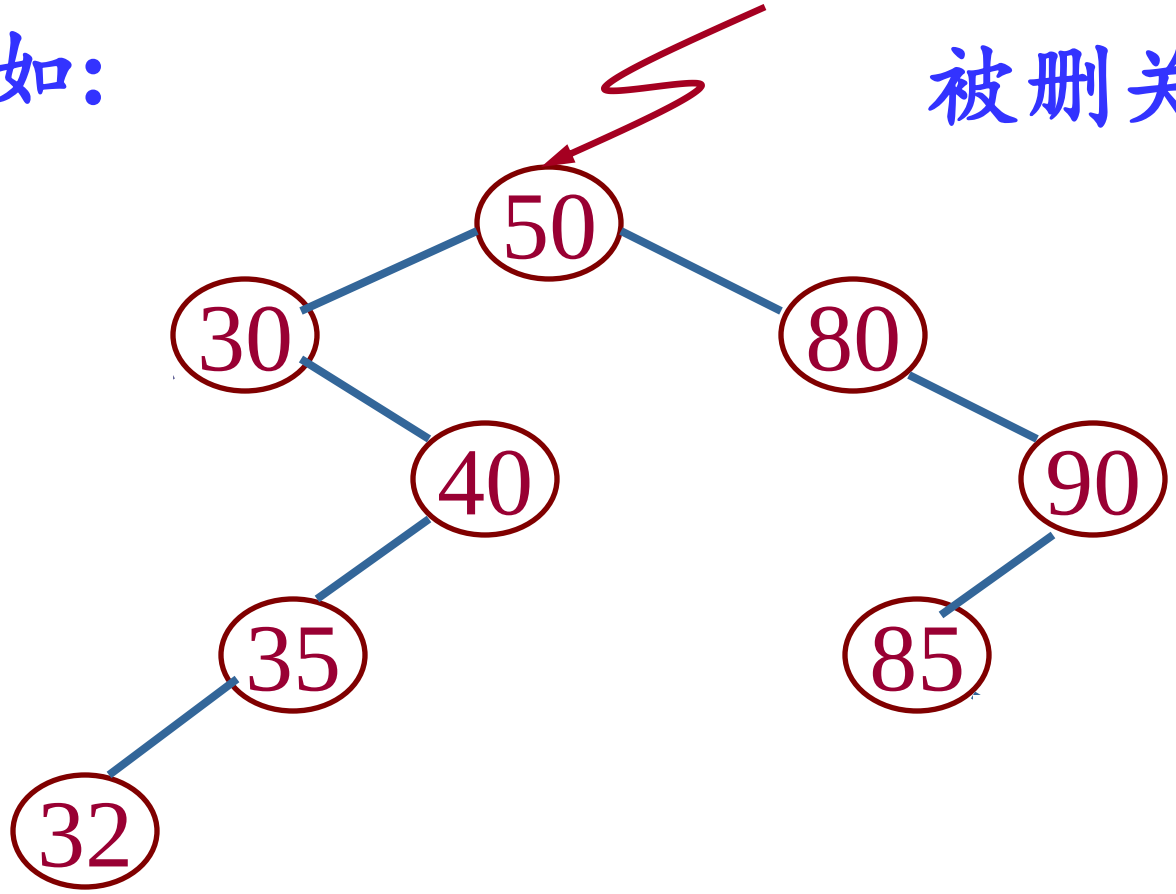
（2）被删除的结点只有左子树或者只有右子树；

（3）被删除的结点既有左子树，也有右子树。

（1）被删除的结点是叶子结点

例如：

被删关键字 = 88

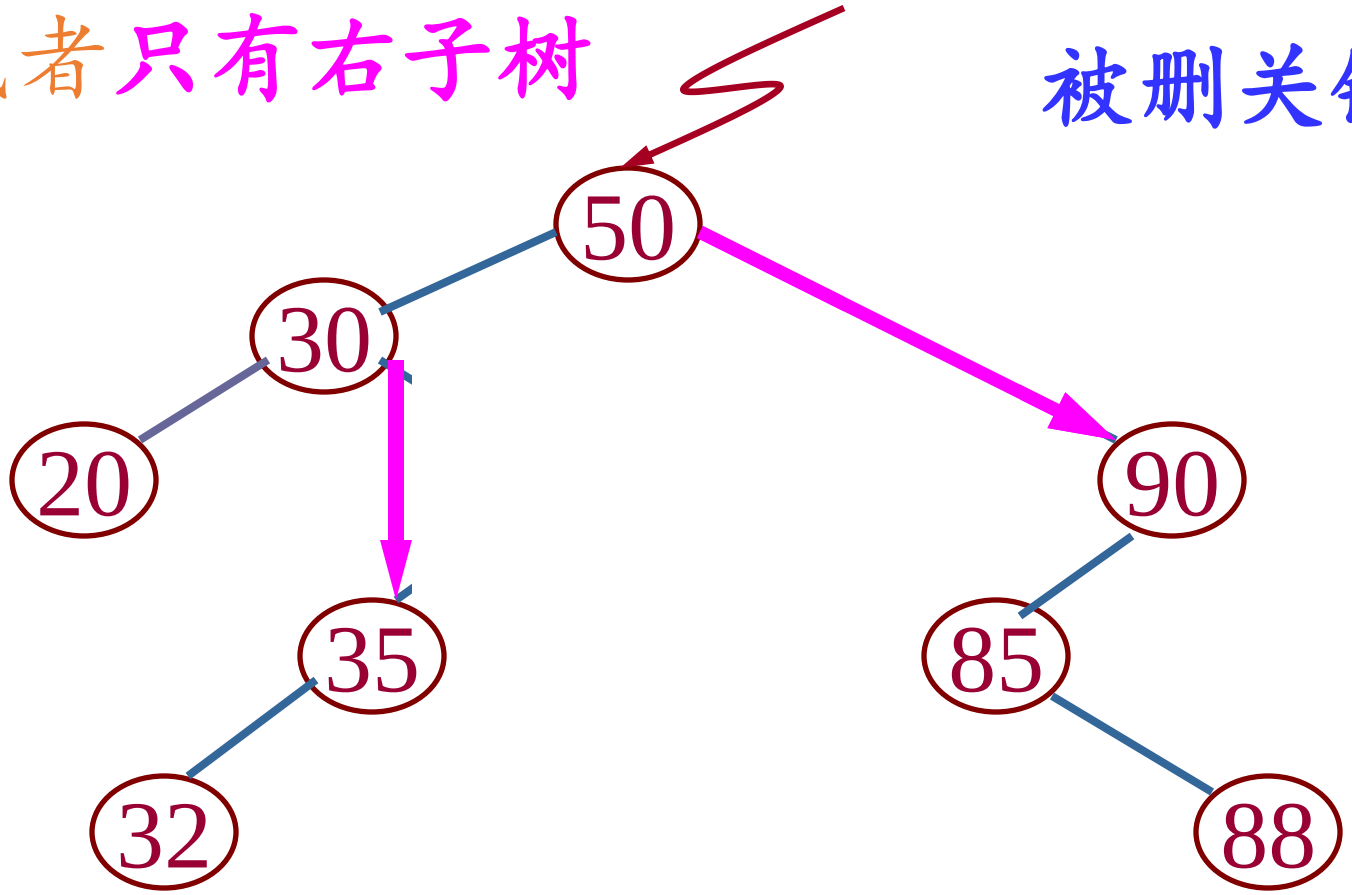


其双亲结点中相应指针域的值改为“空”

（2）被删除的结点只有左子树

或者只有右子树

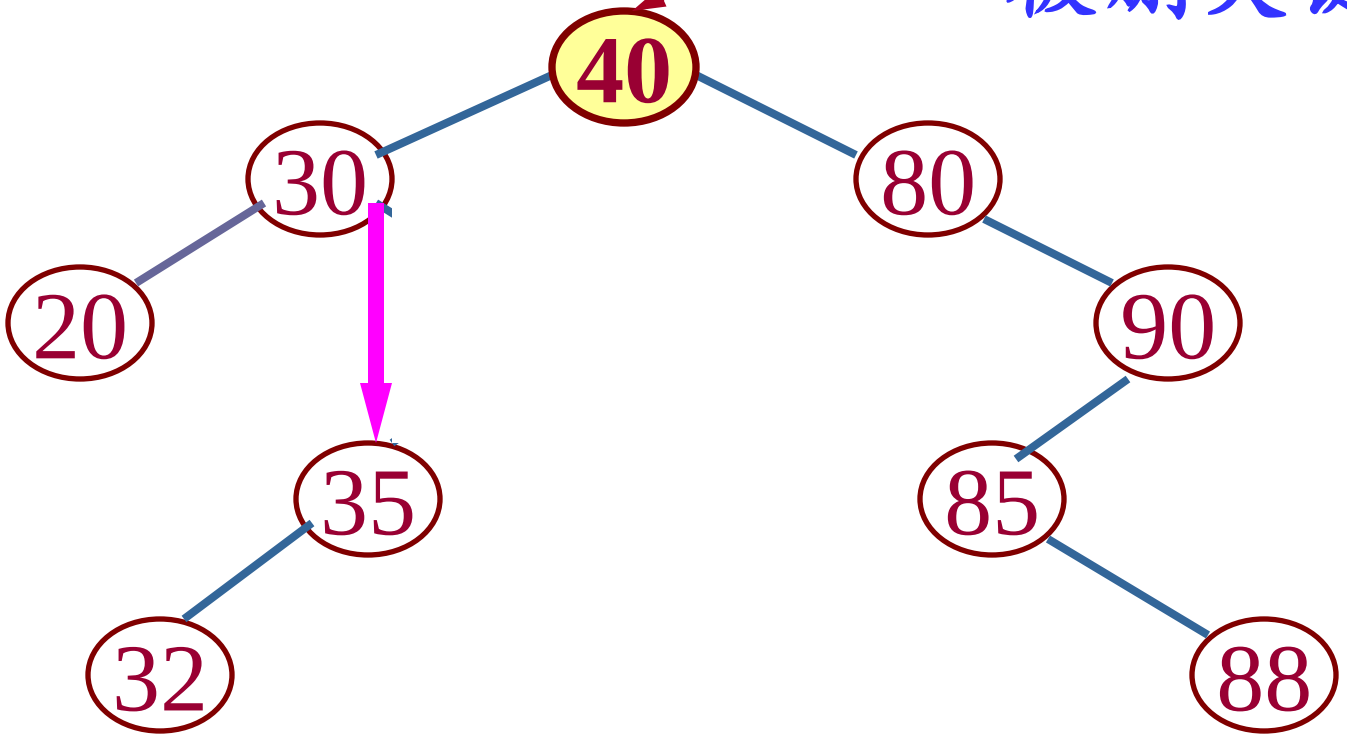
被删关键字 = **80**

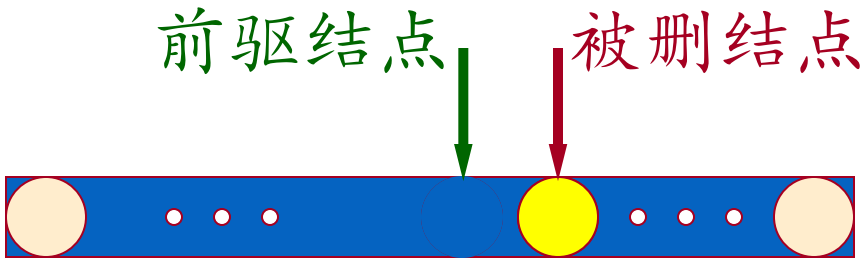


其双亲结点的相应指针域的值改为“指向被删除结点的左子树或右子树”。

（3）被删除的结点既有左子树，也有右子树

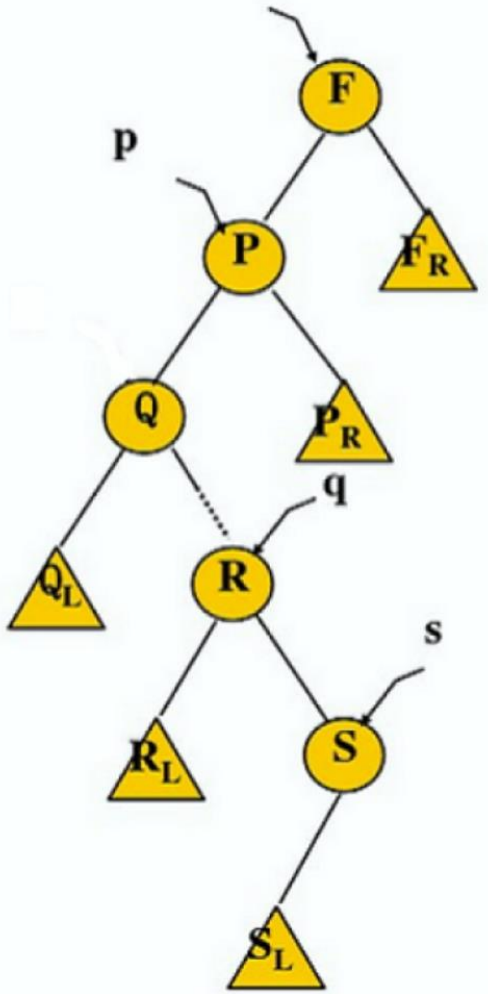
被删关键字 = 50

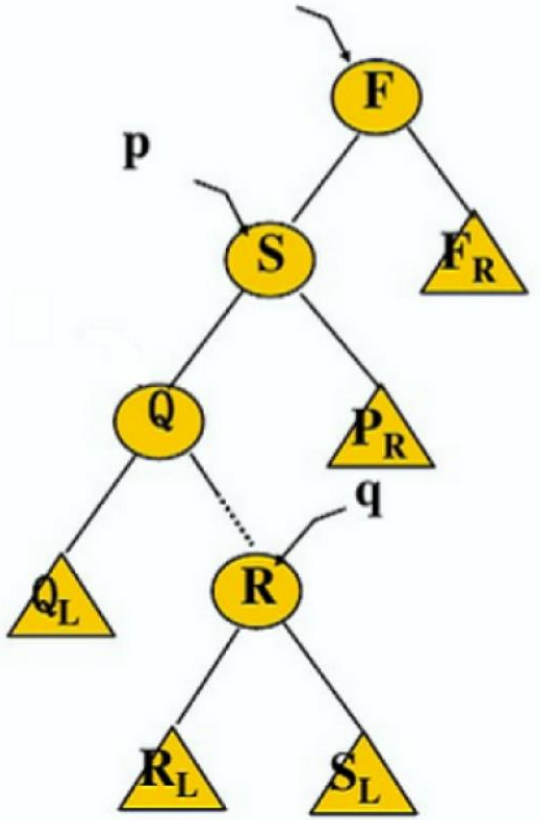




以其前驱替代之，然后再删除该前驱结点

令*p结点的直接前驱(或直接后继)代替*p，然后再从二叉排序树中删除它的直接前驱(或直接后继)。当以直接前驱*s代替*p时，由于*s只有左子树S_L，则在删去*s之后，只要令S_L,为*s的双亲*r的右子树即可。





算法描述如下：

```
Status DeleteBST(BiTree &T, KeyType key){
    // 若二叉排序树T中存在其关键字等于key的
    // 数据元素，则删除该数据元素结点，并返回
    // 函数值TRUE，否则返回函数值FALSE
  if(!T) return FALSE;
    // 不存在关键字等于key的数据元素
  else {……          }
}// DeleteBST
```

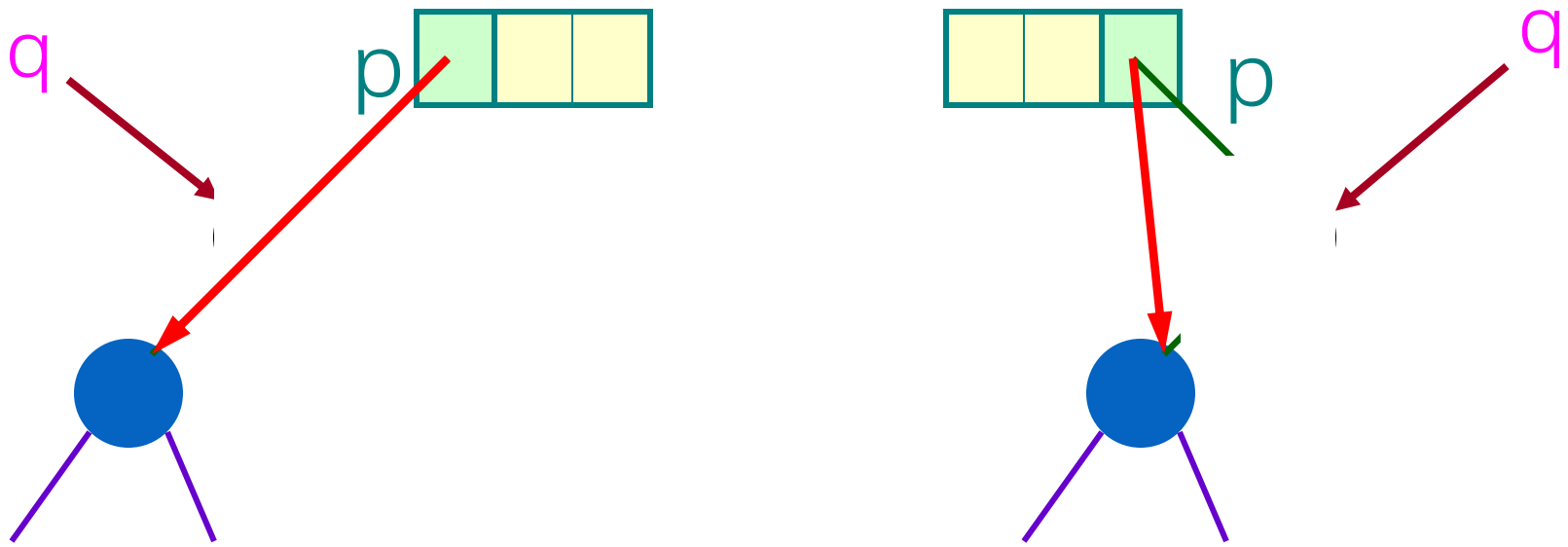
```
if( EQ (key, T->data.key) )
    { Delete (T); return TRUE; }
      // 找到关键字等于key的数据元素
else if( LT (key, T->data.key) )
    DeleteBST( T->lchild, key );
      // 继续在左子树中进行查找
else
    DeleteBST( T->rchild, key );
      // 继续在右子树中进行查找
```

其中删除操作过程如下所描述：

```
void Delete ( BiTree &p ){
    // 从二叉排序树中删除结点p,
    // 并重接它的左子树或右子树
   if(!p->rchild){ …… } //右子树空
   else if(!p->lchild){ …… }//左子树空
   else {  ……  } //左右子树均不空
}// Delete
```

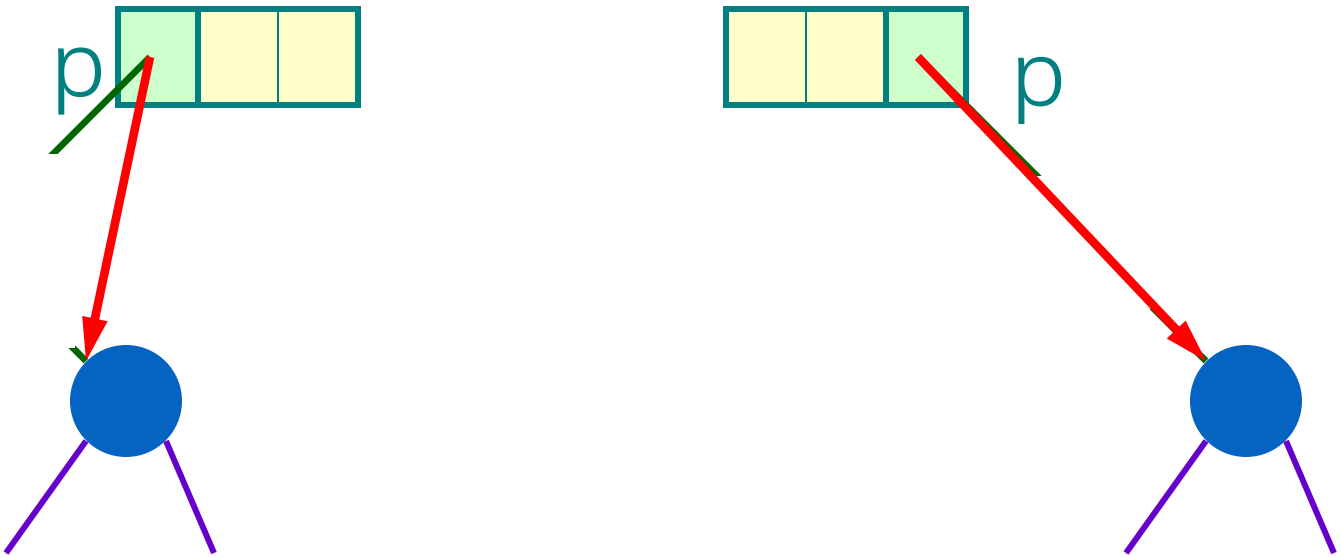
// 右子树为空树则只需重接它的左子树

```
q = p;  p = p->lchild;  free(q);
```



// 左子树为空树只需重接它的右子树

```
q = p;  p = p->rchild; free(q);
```

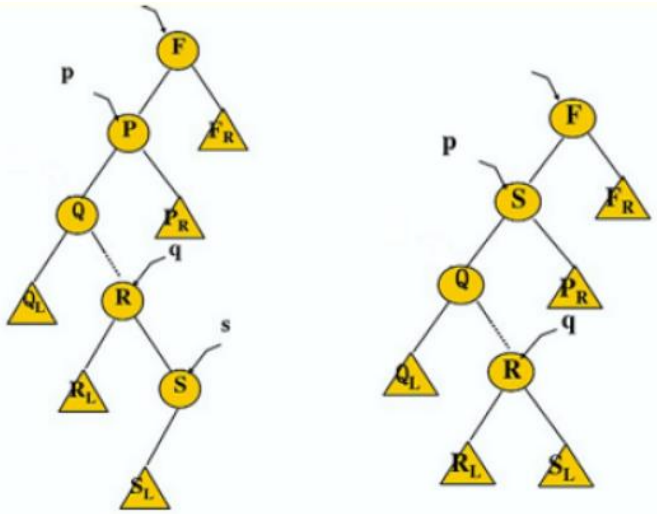


// 左右子树均不空

q = p; s = p->lchild;

while (s->rchild)

{ q = s; s = s->rchild; } //s指向被删结点的前驱



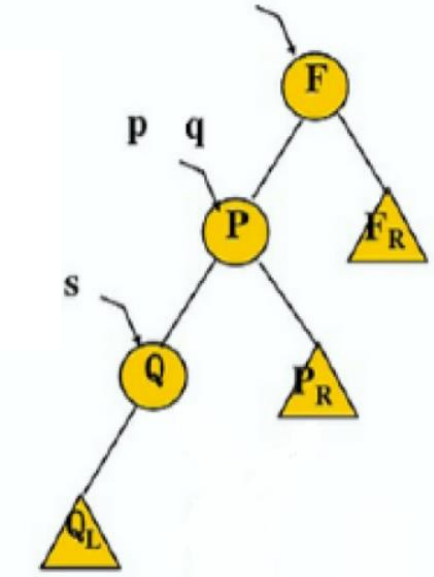
p->data = s->data;

// 重接*q的左子树

if (q **!=** p) q->rchild = s->lchild;

else q->lchild = s->lchild;

free(s);

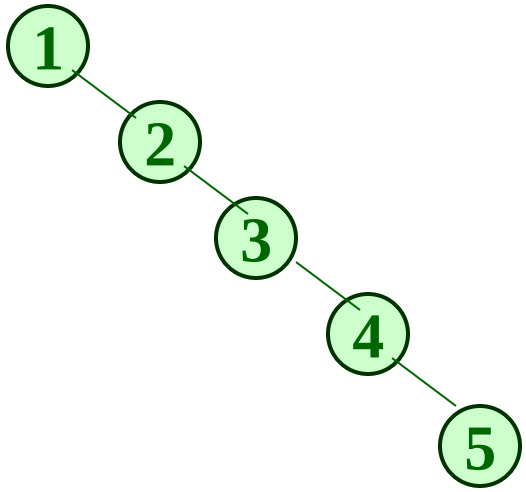


5. 查找性能的分析

对于每一棵特定的二叉排序树，均可按照平均查找长度的定义来求它的 ***ASL*** 值，显然，由值相同的 ***n*** 个关键字，构造所得的不同形态的各棵二叉排序树的平均查找长度的值不同，甚至可能差别很大。

例如：

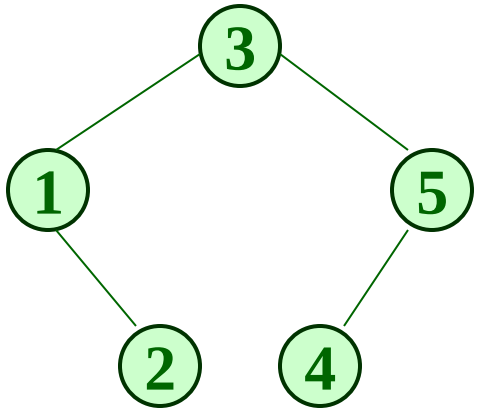
由关键字序列 **1，2，3，4，5** 构造而得的二叉排序树，



$$ASL = (1+2+3+4+5) / 5$$

$$= 3$$

由关键字序列 **3，1，2，5，4** 构造而得的二叉排序树，

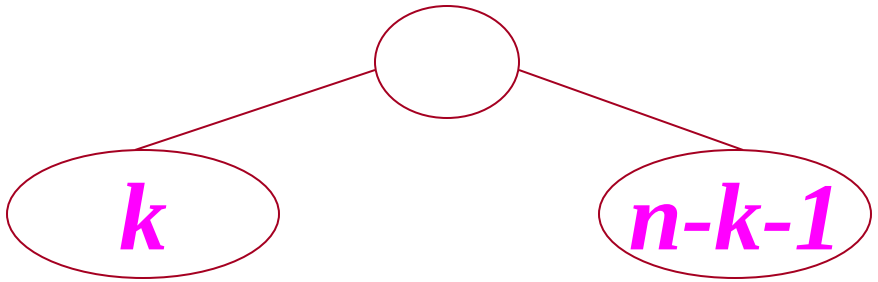


$$ASL = (1+2+3+2+3) / 5$$

$$= 2.2$$

下面讨论平均情况：

不失一般性，假设长度为 ***n*** 的序列中有 ***k*** 个关键字**小于**第一个关键字，则必有 ***n-k-1*** 个关键字**大于**第一个关键字,由它构造的二叉排序树：



的平均查找长度是 *n* 和 *k* 的函数

$$P(n, k) \quad (0 \le k \le n-1)。$$

假设 n 个关键字可能出现的 $n!$ 种排列的可能性相同，则含 n 个关键字的二叉排序树的平均查找长度：

$$ASL = P(n) = \frac{1}{n}\sum_{k=0}^{n-1} P(n,k)$$

在**等概率查找**的情况下，

$$\boldsymbol{P(n,k) = \sum_{i=1}^{n} p_i C_i = \frac{1}{n}\sum_{i=1}^{n} C_i}$$

$$P(n,k) = \frac{1}{n}\sum_{i=1}^{n} C_i = \frac{1}{n}\left(C_{root} + \sum_{L} C_i + \sum_{R} C_i \right)$$

$$= \frac{1}{n}\left(1 + k(P(k)+1) + (n-k-1)(P(n-k-1)+1)\right)$$

$$= 1 + \frac{1}{n}\left(k \times P(k) + (n-k-1) \times P(n-k-1)\right)$$

由此

$$P(n)=\frac{1}{n}\sum_{k=0}^{n-1}\left(1+\frac{1}{n}\left(k\times P(k)+(n-k-1)\times P(n-k-1)\right)\right)$$

$$=1+\frac{2}{n^2}\sum_{k=1}^{n-1}\left(k\times P(k)\right)$$

可类似于解差分方程，此递归方程有解：

$$P(n)=2\frac{n+1}{n}\log n+C$$

二、二叉平衡树

何谓“二叉平衡树”？

如何构造“二叉平衡树”

二叉平衡树的查找性能分析

二叉平衡树

又称AVL树。它或者是一棵空树，或者是具有下列性质的二叉树：它的左子树或右子树都是平衡二叉树，且左子树和右子树的深度之差的绝对值不超过1。

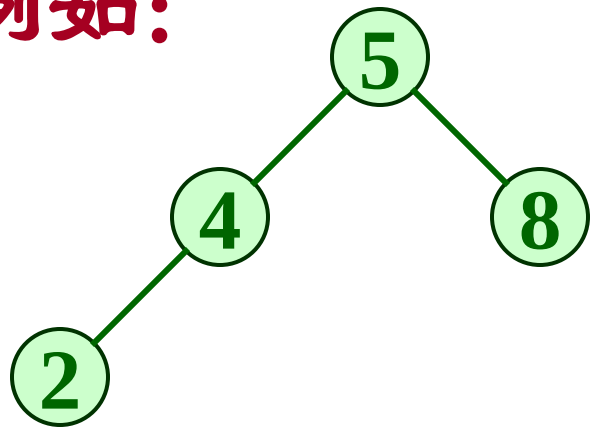
平衡因子

二叉树上任一结点的左子树深度减去右子树深度的差值，称为此结点的平衡因子。

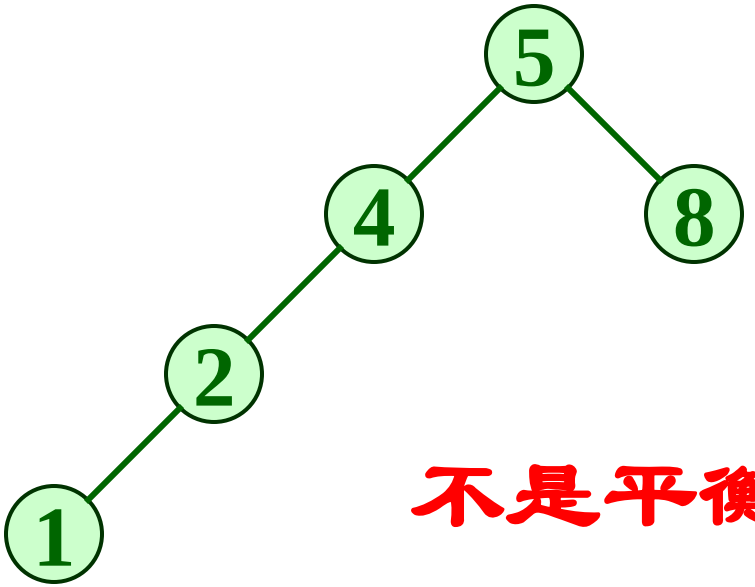
二叉平衡树是二叉查找树的另一种形式，其特点为：

树中每个结点的左、右子树深度之差的绝对值不大于1 $|h_L - h_R| \le 1$ 。

例如：



是平衡树

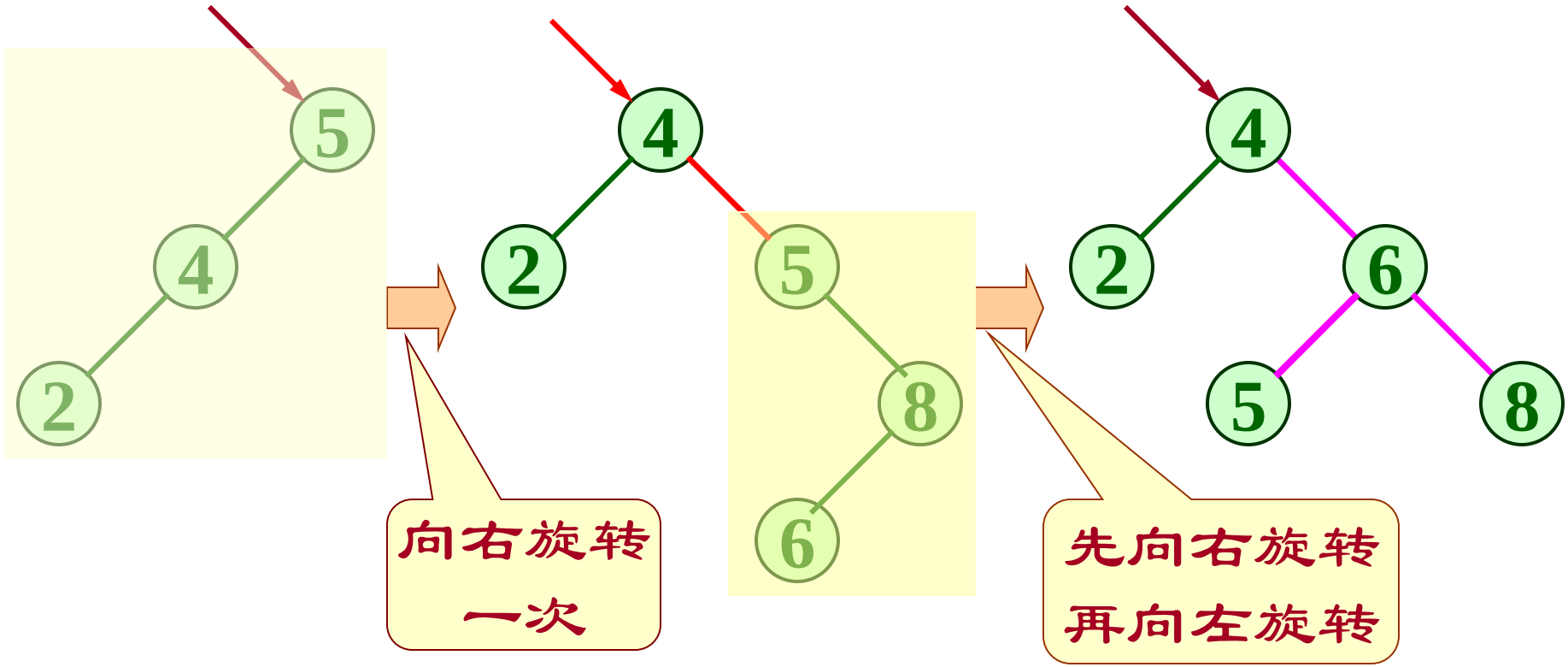


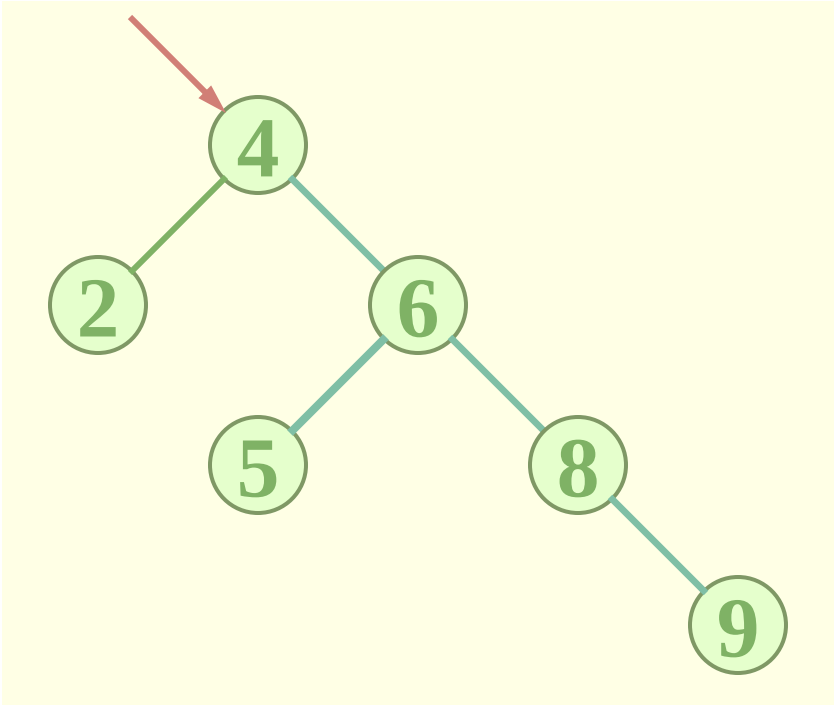
不是平衡树

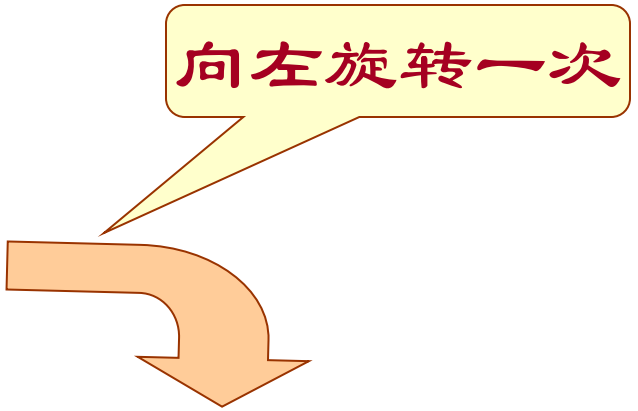
构造二叉平衡（查找）树的方法是：

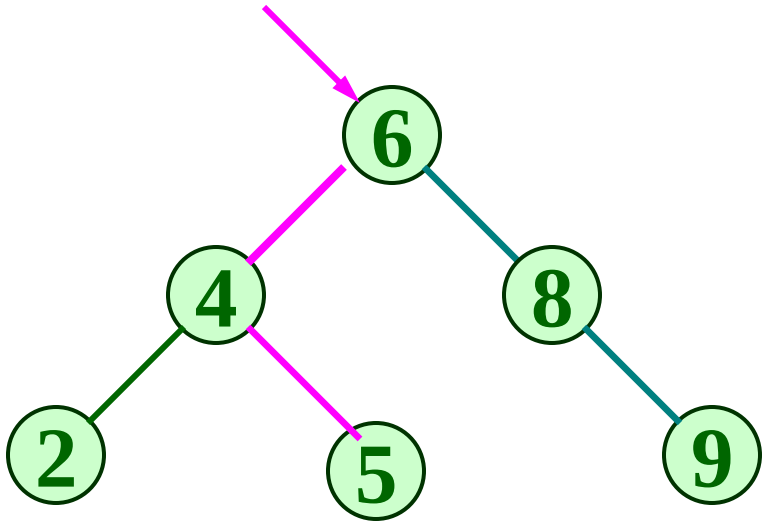
在插入过程中，采用平衡旋转技术。

例如:依次插入的关键字为5, 4, 2, 8, 6, 9

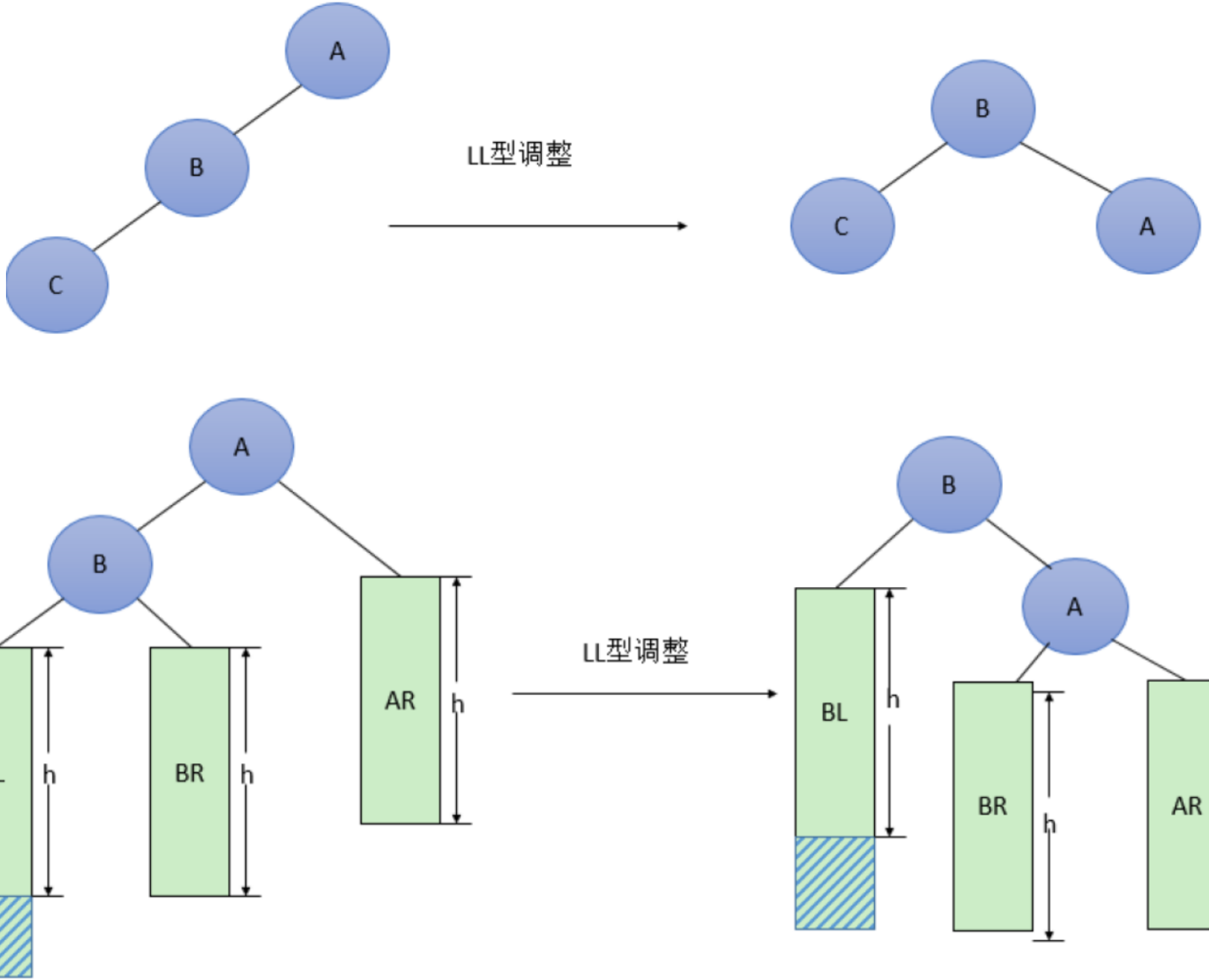


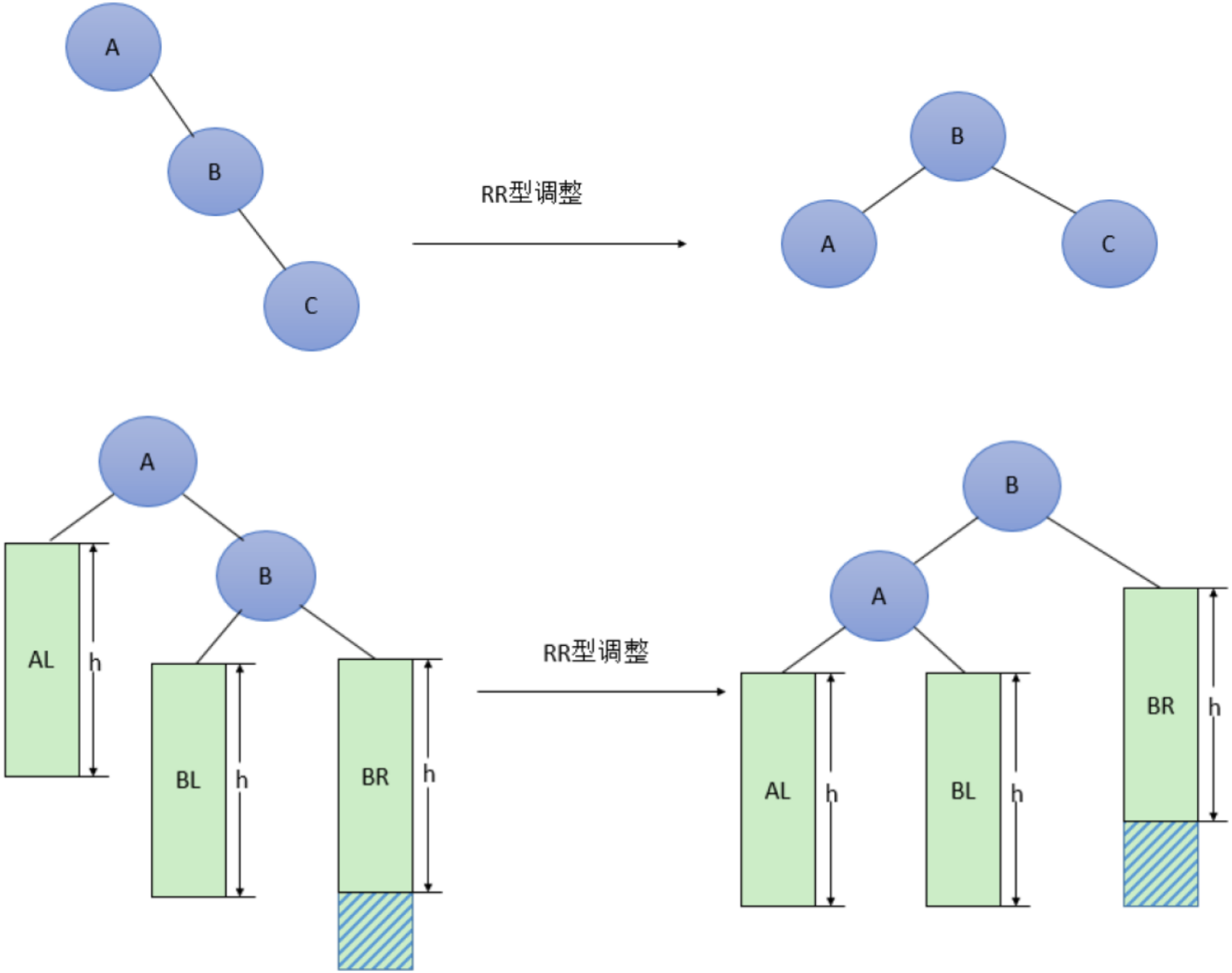


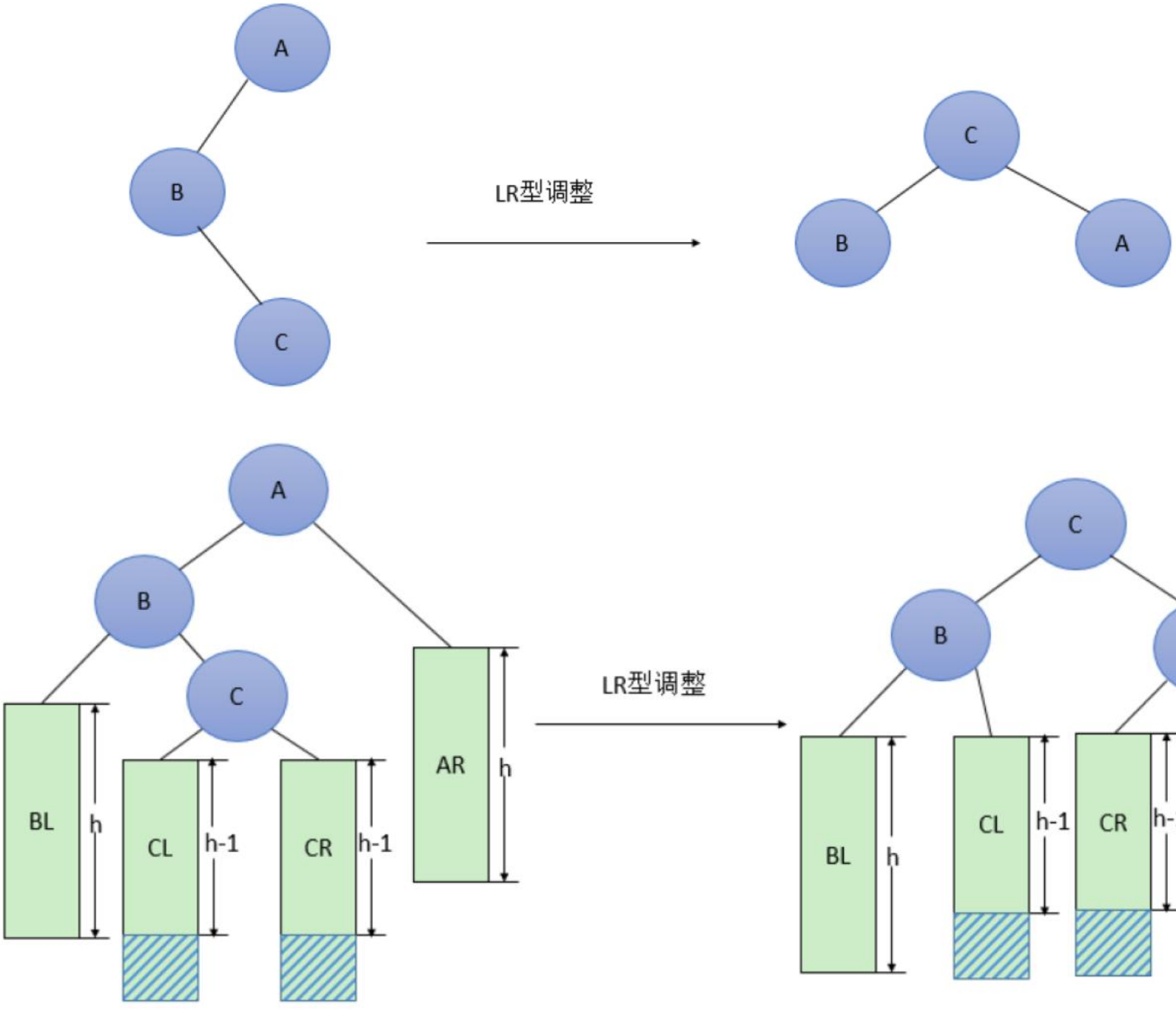


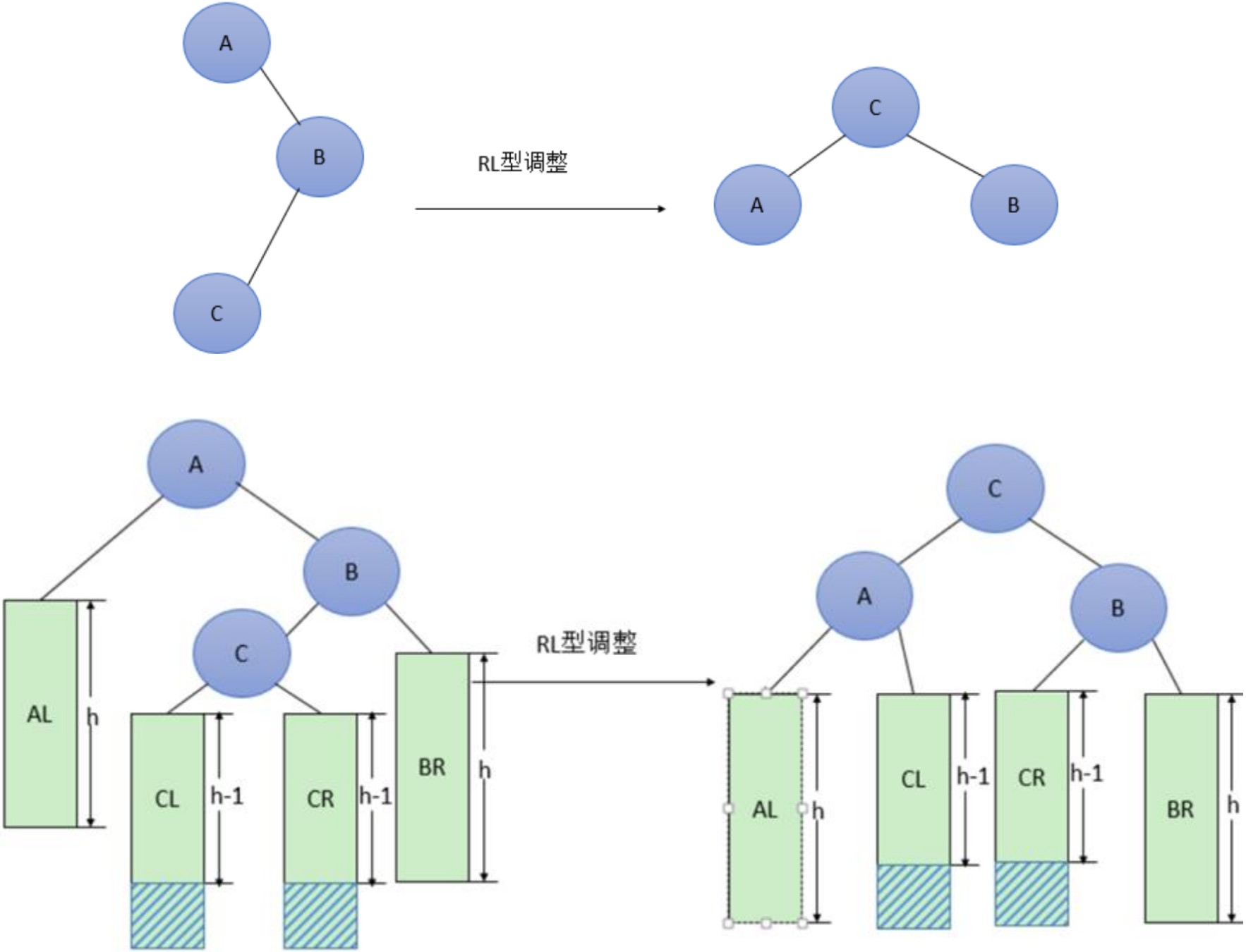


继续插入关键字 9


A
B
C
LL型调整
B
C
A
A
B
BL
h
BR
h
AR
h
LL型调整
B
A
BL
h
BR
h
AR
h


A
B
C
RR型调整
B
A
C
A
B
AL
h
BL
h
BR
h
RR型调整
B
A
AL
h
BL
h
BR
h


A
B
C
LR型调整
C
B
A
A
B
C
BL
h
CL
h-1
CR
h-1
AR
h
LR型调整
C
B
A
BL
h
CL
h-1
CR
h-1
AR
h


A
B
C
RL型调整
C
A
B
A
B
C
AL
h
CL
h-1
CR
h-1
BR
h
RL型调整
C
A
B
AL
h
CL
h-1
CR
h-1
BR
h

第十二次作业：

设查找的关键字序列为：

{36, 20, 59, 32, 67, 18, 90, 48, 75, 39, 15, 42, 12}

画出通过逐个节点的插入操作所生成的平衡二叉树

平衡树的查找性能分析：

在平衡树上进行查找的过程和二叉排序树相同，因此，查找过程中和给定值**进行比较的关键字的个数**不超过平衡树的深度。

问：含 ***n*** 个关键字的二叉平衡树可能达到的**最大深度**是多少？

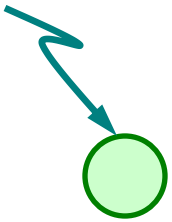
先看几个具体情况：

$n = 0$

空树

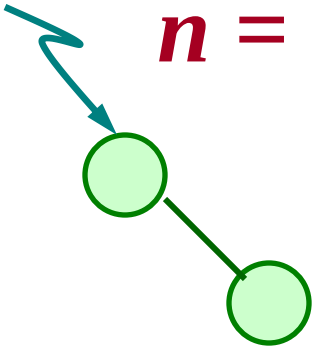
最大深度为 0

$n = 1$



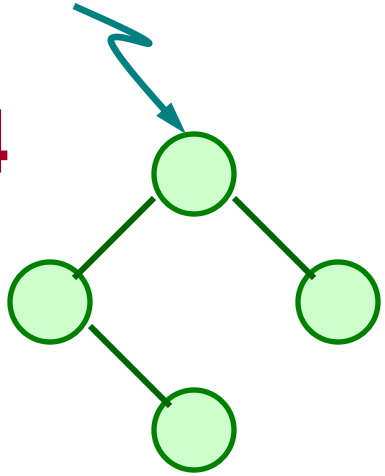
最大深度为 1

$n = 2$



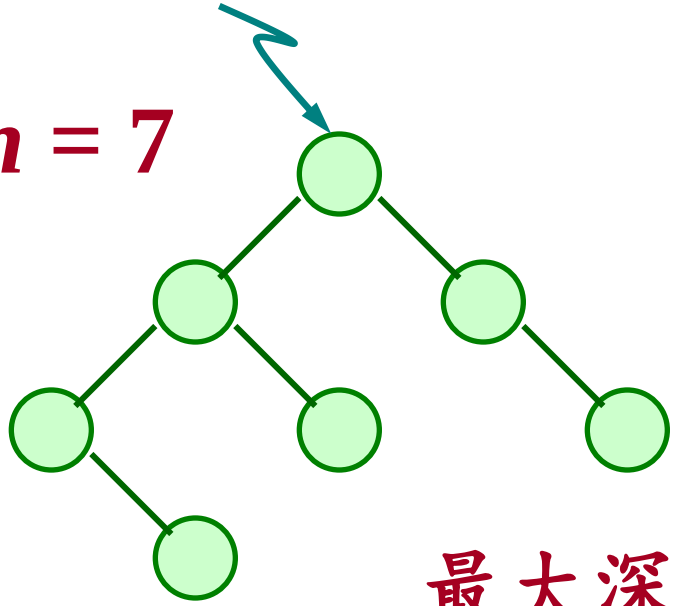
最大深度为 2

$n = 4$



最大深度为 3

$n = 7$



最大深度为 4

反过来问，**深度为 h** 的二叉**平衡树**中所**含结点的最小值 N_h** 是多少？

$h = 0$　　$N_0 = 0$　　$h = 1$　　$N_1 = 1$

$h = 2$　　$N_2 = 2$　　$h = 3$　　$N_3 = 4$

一般情况下　　$N_h = N_{h-1} + N_{h-2} + 1$

利用归纳法可证得　　$N_h = F_{h+2} - 1$

由此推得，深度为 h 的二叉平衡树中所含结点的最小值 $N_h = \varphi^{h+2}/\sqrt{5} - 1$。

反之，含有 n 个结点的二叉平衡树能达到的最大深度 $h_n = \log_\varphi(\sqrt{5}(n+1)) - 2$。

因此，在二叉平衡树上进行查找时，查找过程中和给定值进行比较的关键字的个数和 $log(n)$ 相当。

本章学习要点

① 掌握顺序表和有序表的查找方法及其平均查找长度的计算方法。

② 掌握静态查找树的构造方法和查找算法、索引顺序查找方法，理解静态查找树和折半查找的关系。

③ 熟练掌握二叉排序树、二叉平衡树的构造和查找方法，理解二叉排序树的插入删除算法。